Automata

* Language :- Set of strings (words)

* string : is a sequence of symbols

* set :- is a collection of elements.

List Vs. Set

* لا يجوز التكرار في الـ set

* Set operations :-

1) $1 \in A$ (Belongs to)
2) Union
3) Difference
4) Intersection
5) Complement
6) set cardinality
7) Power sets
8) cartesion
9) Product

Set Representation:

finite

Ex. $A=\{a,b,c,d,e,f\}$

$\equiv$

$A=\{a,b,\ldots,f\}$

infinite

Ex. $B=\{2,4,6,\ldots\}$ for Even No.

$\equiv$

$B=\{2k : k>0\}$

Ex.

$C=\{1,3,5,\ldots\}$ for odd No.

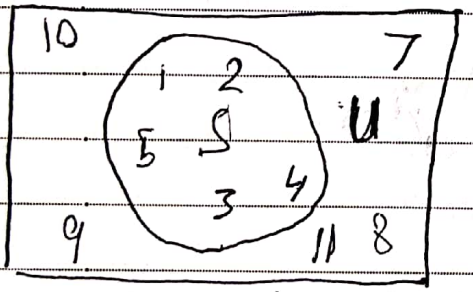
$\equiv$

$C=\{2k+1 : k\geq 0\}$

Rule (Pattern)

---

* Universal set: all possible elements.



* $U \cap S = S$
* $U \cup S = U$
* $S \subseteq U$
* $\bar{S} = U - S$

Ex. given $A=\{a,b,d\}$ $B=\{a,c,f\}$ find:-

1. $A \cup B = \{a,b,d,c,f\}$
2. $A \cap B = \{a\}$
3. $A-B=\{b,d\}$
4. $B-A=\{c,f\}$
5. $\bar{A} = U-A$

Note:- $\bar{A} = A'$ (complement)

6. $|A|=3$
7. $|A \cup B| = 5$

* Subset :- $\subset$ OR $\subseteq$

Ex. $A = \{2, 5\}$ $B = \{1, 2, 3, 4, 5\}$

$\Rightarrow A \subseteq B$

* Null set / empty set :- $\emptyset$ OR $\{\}$

$\emptyset = \{x : x \neq x\}$

* Power sets :- 2^S

def:- $|2^S| = 2^{|S|}$

$|S| \leftarrow$ set size

Ex. given $A = \{1, 2\}$ find 2^A

$2^A = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$

$|2^A| = 2^{|A|} = 2^2 = 4$

Ex. $S = \emptyset$ find 2^S :- $2^S = \{\emptyset\} \Rightarrow |2^S| = 2^{|S|} = 2^0 = 1$

Ex. $C = \{a, b, c\}$

$2^C = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$

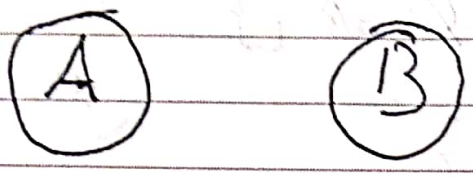
Notes :- $\emptyset = \{\}$, $S \cup \emptyset = S$, $S \cap \emptyset = \emptyset$

$S - \emptyset = S$, $\emptyset - S = \emptyset$, $\overline{\emptyset} = U$

* Disjoint sets :-

$A = \{1, 2\}$, $B = \{3, 4\}$

$A \cap B = \emptyset$



* Cartesian Product :- $A \times B$

* $A = \{1, 2\}$, $B = \{2, 3, 4\}$

$A \times B = \{(1,2), (1,3), (1,4), (2,2), (2,3), (2,4)\}$

$|A \times B| = |A| * |B| = 2 * 3 = 6$

* Language is a set of strings

* string :- sequence of symbols from alphabet.

* alphabet : set of symbols. (Σ)

$\Sigma_{en} = \{a, b, c, \cdots, z, A, B, C, \cdots, Z\}$

* String operations

[1] concatenation (•)

$S_1 = a_1 a_2 a_3 \cdots a_n$, $S_2 = b_1 b_2 b_3 \cdots b_m$

* $S_1 \cdot S_2 = a_1 a_2 a_3 \cdots a_n b_1 b_2 b_3 \cdots b_m$

* $S_2 S_1 = b_1 b_2 b_3 \cdots b_m a_1 a_2 a_3 \cdots a_n$

Ex. given $\Sigma = \{0, 1\}$, $S_1 = 001$, $S_2 = 110$ find :-

[1] $S_1 \cdot S_1 = 001001$ [2] $S_2 S_1 = 110001$

[3] $S_1 S_2 S_1 = 001110001$

* Reverse (S^R)

$w = a_1 a_2 a_3 \cdots a_n \Rightarrow w^R = a_n a_{n-1} a_{n-2} \cdots a_3 a_2 a_1$

Ex find $(abbaba)^R \Rightarrow abbaba$

Strings

string operations on

[1] Concatenation [2] Reverse

[3] String length ||

Ex $w = a_1 a_2 a_3 \dots a_n \Rightarrow |w| = n$

Ex $|ab| = 2$ $|a| = 1$

$|01001| = 5$

* length of concatenation

$|wv| = |w| + |v|$

* empty string "λ" by def $|\lambda| = 0$

[4] Exponent operation

$w^n = \underbrace{www\dots}_{n}$ by def $w^0 = \lambda$

Ex $w = ab$

[1] $w^0 = \lambda$ [2] $w^1 = ab$ [3] $w^2 = abab$

Substring

string	substring
a b a a b b b	b
a b a a b b b	a b
a b a a b b b	a b b

* Prefix and Suffix

$w = uv$

prefix ↙ ↘ suffix

Ex. String "01101"

Prefix	Suffix
λ	01101
0	1101
01	101
011	01
0110	1
01101	λ

* the * Operation (Kleene-closure)

def: the set of all possible strings from alphabet Σ.

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cdots \Sigma^\infty$ $\quad \Sigma = \{a, b\}$

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$$

Ex. given $\Sigma = \{1\}$ find Σ^*

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cdots$

$\Sigma^0 = \{\lambda\}$ $\quad$ $\Sigma^1 = \{1\}$

$\Sigma^2 = \{11\}$ $\quad$ $\Sigma^3 = \{111\}$

$\Rightarrow \Sigma^* = \{\lambda, 1, 11, 111, 1111, \ldots\}$

Ex. $\Sigma = \{a, b\}$ find Σ^*

$\Sigma^0 = \{\lambda\}$ $\quad$ $\Sigma^1 = \{a, b\}$

$\Sigma^2 = \{aa, ab, ba, bb\}$

$\Sigma^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, aba, \cdots, bbb, aaaa, \cdots, bbbb, \cdots\}$

Ex. given $\Sigma = \{+, *\}$ find Σ^*?

$\Sigma^0 = \{\lambda\}$ $\quad$ $\Sigma^1 = \{+, *\}$

$\Sigma^2 = \{++, +*, *+, **\}$

$\Sigma^3 = \{+++, *++, +*+, +**, *++, *+*, **+, ***\}$

$\Sigma^* = \{\lambda, +, *, ++, +*, *+, **, +++, \ldots, ***, ++++, \ldots, ****, \ldots\}$

Ex. given $\Sigma = \{a, b, c, d, \ldots, z\}$ find Σ^*

aab

$\Sigma^* = \{\lambda, a, b, \ldots, z, aa, ab, \ldots, zz, aaa, \ldots, zzz, aaaa, aaab, \ldots, zzzz, \ldots\}$

the + operation (Positive-closure)

def: the set of all possible strings from alphabet Σ exept λ.

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cdots \Sigma^\infty$$

therefore $\boxed{\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i}$ $\qquad \Sigma^+ = \Sigma^* - \{\lambda\}$

* language is any subset of Σ^*

$\Sigma = \{a, b\}$

Languages

1. $\{a, b\}$ 2. $\{ba, bb, aaa\}$

3. $\{\lambda\}$ 4. $\{\}$

Examples

1. $\Sigma = \{1\}$

$L = \{1^n : n \geq 1\} = \{1, 11, 111, 1111, \cdots\} \subseteq \Sigma^*$

2. $L = \{1^{2n} : n \geq 0\} = \{\lambda, 11, 1111, 111111, \cdots\} \subseteq \Sigma^*$

[3] $\Sigma = \{a, b\}$

— $L_1 = \{a^n b^n : n \geq 0\}$

* $a^0 b^0 = \lambda\lambda = \lambda$
* $w\lambda = w$ * $\lambda w = w$
* $\lambda w = w$ * $\lambda\lambda w\lambda = w$

$\Rightarrow \{\lambda, ab, aabb, aaabbb, \ldots\}$

[4] $L_2 = \{a^{3^n} b : n \geq 1\} = \{aaab, aaaaaaab, \ldots\}$

[5] $L_3 = \{a^{2n} b^{3k} : n \geq 0, k \geq 1\} = \{bbb, bbbbbb, \ldots, aabbb,$

$aabbbbbb, \ldots, aaaabbb, aaaabbbbbb, \ldots\}$

* take 'n' with cases of 'k', then increment n like nested loop.

Ex Find the Rule (Pattern) of the following languages.

[1] $L = \{aba, abba, abbba, \ldots\}$

$L_{Rule} = \{ab^n a : n \geq 1\}$

[2] $L = \{bb, abaabb, aabaaaabbb, \ldots\}$

$L_{Rule} = \{a^n b a^{2n} b^{(n+1)} : n \geq 0\}$

Ex alphabet $\Sigma = \{0, 1, 2, \ldots, 9\}$

Language:

$L_{primes} = \{x : x \in \Sigma^* \text{ and } x \text{ is prime}\}$
$= \{2, 3, 5, 7, 11, 13, 17, 19, \ldots\}$

$L_{even} = \{x : x \in \Sigma^* \text{ and } x \text{ is even}\}$
$= \{2, 4, 6, 8, \ldots\}$

$L_{odd} = \{x : x \in \Sigma^* \text{ and } x \text{ is odd}\}$
$= \{3, 5, 7, 9, \ldots\}$

Ex. $\Sigma = \{1, \#\}$

$L_{\text{squares}} = \{x\#y : x = 1^n, y = 1^m, n \geq 1, m = n^2\}$

$= \{"1\#1", "11\#1111", "111\#111111111", \ldots\}$

$\{\} \neq \{\lambda\} \Rightarrow |\{\}| = 0 \;/\; |\{\lambda\}| = 1$

$|\lambda| = 0$

Language Operations: on $\Sigma = \{a, b\}$

* $\{a, ab, aaaa\} \cup \{bb, ab\} \Rightarrow \{a, ab, bb, aaaa\}$

* $\{a, ab, aaaa\} \cap \{bb, ab\} \Rightarrow \{ab\}$

* $\{a, ab, aaaa\} - \{bb, ab\} \Rightarrow \{a, aaaa\}$

* $\{a, ab\} = \Sigma^* - \{a, ab\} \Rightarrow \{\lambda, b, aa, ba, bb, aaa, \ldots\}$

* $L^R = \{w^R : w \in L\}$

Ex $\{abb, baaba, aaabab\}^R \Rightarrow \{bba, abaab, babaaa\}$

Ex. $L = \{a^n b^{2n} c : n \geq 1\}, \Sigma = \{a, b, c\}$

$L^R = \{c b^{2n} a^n : n \geq 1\}$

* Concatenation:

$$L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$$

Ex. $\{ab, bb, aaa\} \cdot \{1, 2, 3\}$

$\Rightarrow \{ab1, ab2, ab3, bb1, bb2, bb3, aaa1, aaa2, aaa3\}$

Ex. $\{ab, bb\} \cdot \{\lambda, a\} = \{ab, aba, bb, bba\}$

Ex. $L_1 = \{a^n b : n \geq 1\}$ / $L_2 = \{bba^n : n \geq 0\}$

$L_1 L_2 = \{a^x bbba^y : x \geq 1, y \geq 0\}$

$L_1 L_1 = \{a^i b a^j b : i, j \geq 1\}$

$L_2^R L_2 L_1 = \{a^x bbbba^y a^z b : x, y \geq 0, z \geq 1\}$

* $L^n = \underbrace{LLL \cdots L}_{n}$ by def $L^0 = \{\lambda\}$, $L^1 = L$

Ex $L = \{a, ab\}$

$L^3 = \{a, ab\} \cdot \{a, ab\} \cdot \{a, ab\}$

$\Rightarrow \{aaa, aaab, aaba, aabaab, abaa, abaaab, ababa, ababab\}$

* Kleene-Closure (*) $L^* = L^0 \cup L^1 \cup L^2 \cdots = \bigcup_{i=0}^{\infty} L^i$

$\{ab, ba\}^* = \{\lambda, ab, ba, abab, abba, baab, baba, \ldots\}$

$\{ab, ba\}^0 = \{\lambda\}, \{ab, ba\}^1 = \{ab, ba\}, \ldots$

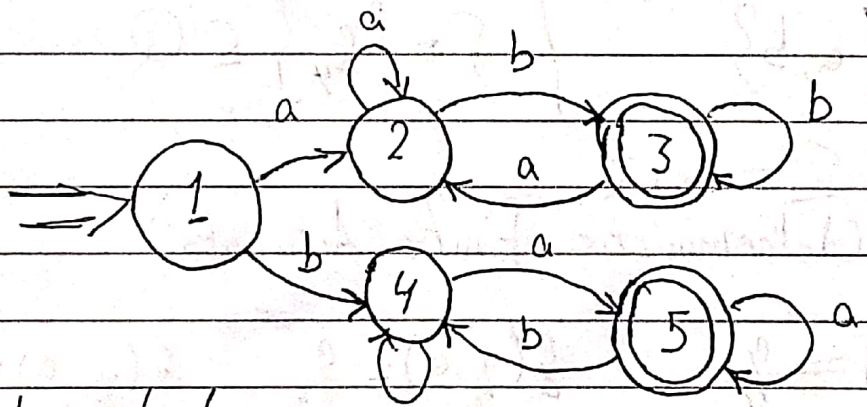
* Positive-Closure (+) $L^+ = L^1 \cup L^2 \cup L^3 \cdots = \bigcup_{i=1}^{\infty} L^i$

$\{\lambda, ab\}^+ = \{\lambda, ab, abab, ababab, \ldots\}$

* Finite Automata

input (string) → [Finite Automata] → output (yes | No) accept / reject

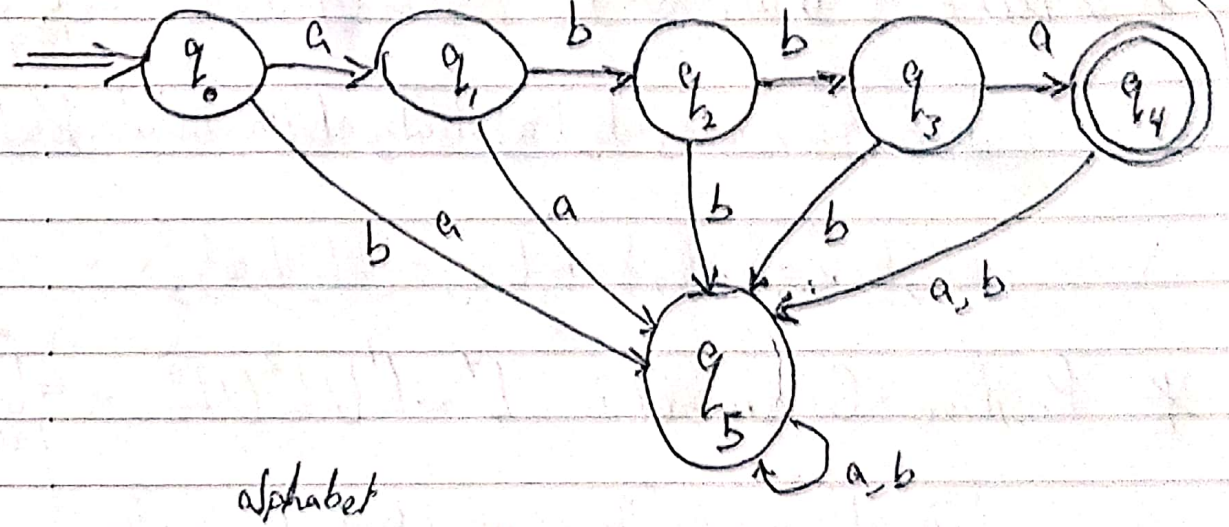
Ex.



input	output
ab	accept
ba	accept
aaa	reject
bbb	reject
aab	accept

i	o
bba	accept
abbb	accept
ababa	reject
baaaba	accept
λ	reject

i	o
a	reject
b	reject



alphabet

$M = (Q, \Sigma, \delta, q_0, F)$ } formal def.

Q = set of states

δ = transition function

q_0 = start state

F = final state(s)

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$ | $q_0 = q_0 \in Q$

$\Sigma = \{a, b\}$ | $F = \{q_4\} \subseteq Q$

$\delta : Q \times \Sigma \longrightarrow Q$

(deterministic Finite Automata)

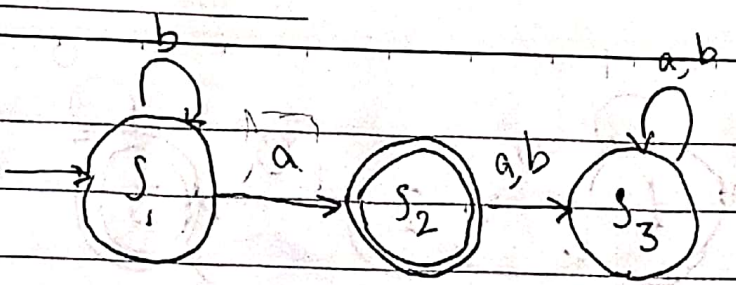
$\delta(q_0, a) = q_1$ $\quad \delta(q_2, a) = q_5$ $\quad \delta(q_4, a) = q_5$

$\delta(q_0, b) = q_5$ $\quad \delta(q_2, b) = q_3$ $\quad \delta(q_4, b) = q_5$

$\delta(q_1, a) = q_5$ $\quad \delta(q_3, a) = q_4$ $\quad \delta(q_5, a) = q_5$

$\delta(q_1, b) = q_2$ $\quad \delta(q_3, b) = q_5$ $\quad \delta(q_5, b) = q_5$

Ex. M:



Find $L(M) = \{a, ba, bba, \dots\}$

Pattern $\Rightarrow \{b^n a : n \geq 0\}$

Notes :-

→ Language name / symbol

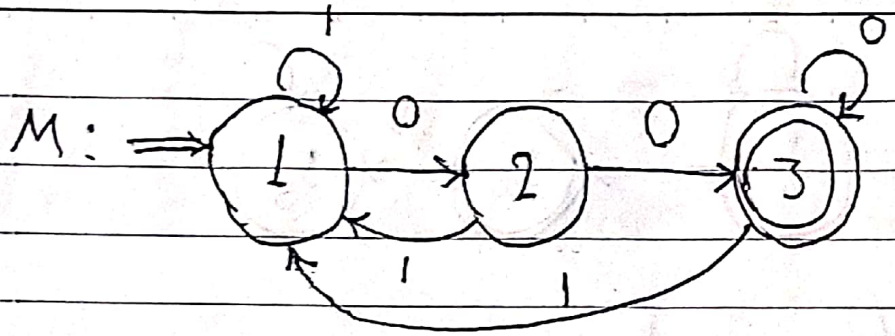
* $L(M)$: Language accepted by M. (the string that gives accepted)

* There can be more than final state

* if first node is an accepted node, the string that gets accepted is 'λ'

Ex.

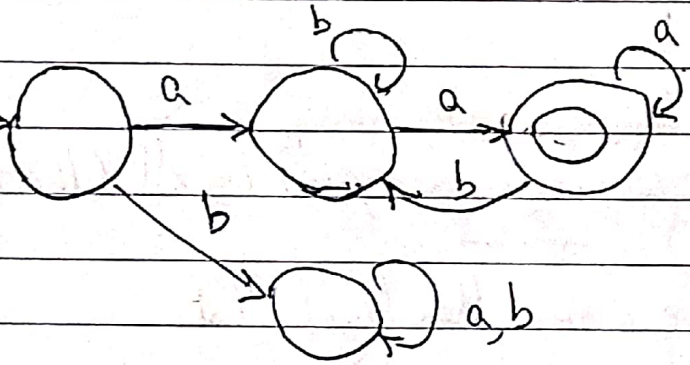
M:



$L(M) = \{00, 000, 100, 0000, 0100, 1000, 1100, \ldots$

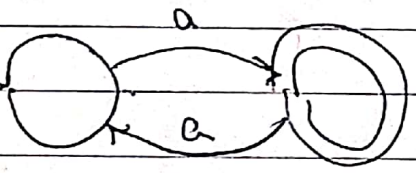
$= \{w00 : w \in \Sigma^*, \Sigma = \{0,1\}\}$

Ex



$L(M) = \{awa : w \in \Sigma^*, \Sigma = \{a,b\}\}$

Ex: M:

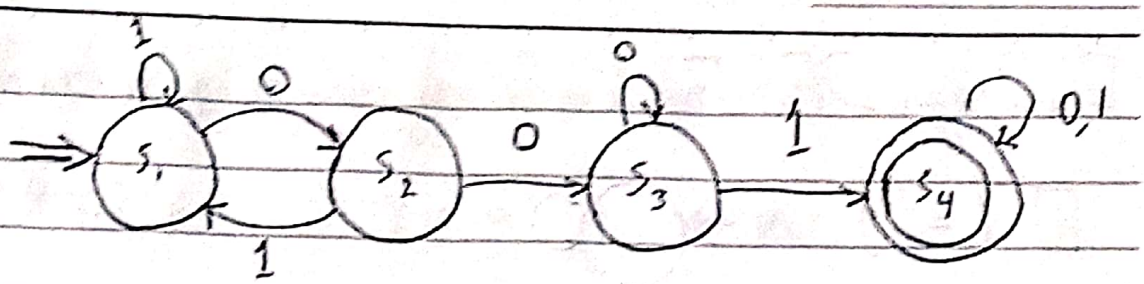


$L(M) = \{w : w \in \{a\}^*, |w| \text{ is odd } \}$

$|w| \bmod 2 = 1 \}$

$\omega 001 \omega$

Ex.



$L(M) = \{001, 0001, 0010, 00111, 1001, 10001, 10011, 10010, 00001, 00100, 00111, \cdots\}$

$= \{\omega : \omega \in \Sigma^* \text{ and } \delta^*(q_0, \omega) \in F\}$

* Note :- all accepted string have "001" as a substring in them.

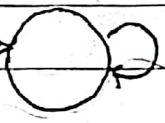
* a string that doesn't contain "001" will be rejected

$\Rightarrow L(M) = \{$ all binary strings that containing "001" as a substring $\}$

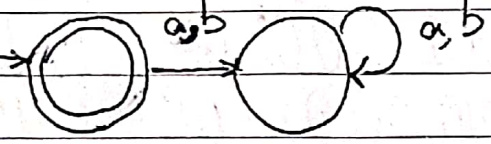
$\Rightarrow \{L(M) = \{\omega_1 001 \omega_2 \circ \omega_1, \omega_2 \in \Sigma^*\}\}$

* Regular Language : if there's a DFA language that is accepted is called a Regular Language

Ex. Design DFA For the following Languages (assume $\Sigma = \{a, b\}$)

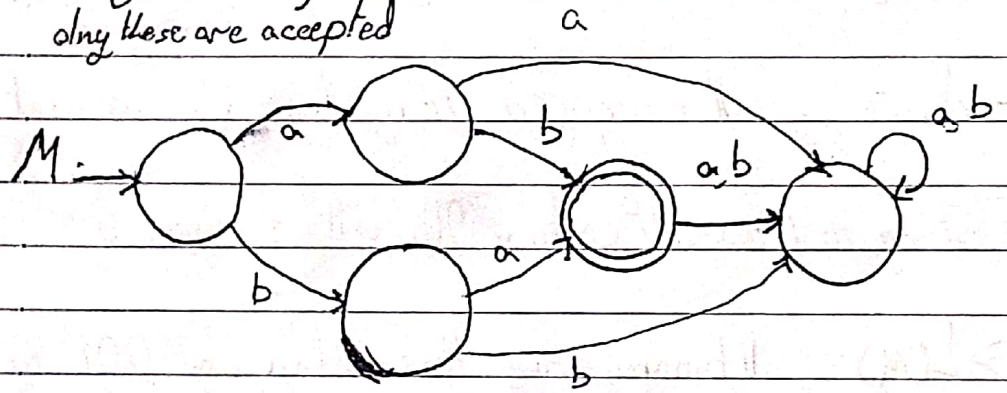
1. $L = \{\}$: 

$\{\} \neq \{\lambda\}$

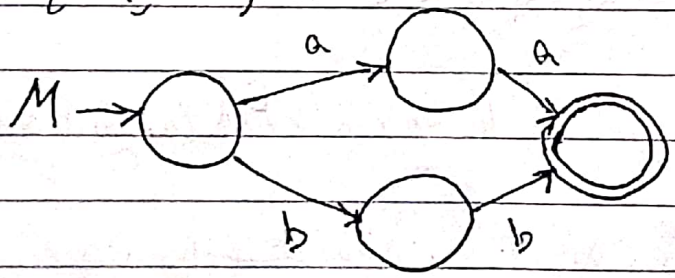
2. $L = \{\lambda\}$: 

3. $L = \{ab, ba\}$

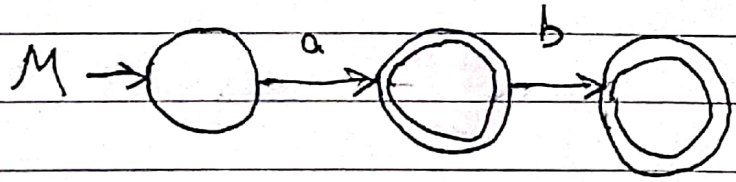
only these are accepted



4. $L = \{aa, bb\}$



5. $L = \{a, ab\}$



⊛ DFA (Deterministic Finite Automata)

$M = (Q, \Sigma, \delta, q_0, F)$

δ → Transition Function

$F \subseteq Q$

$q_0 \in Q$

$$\delta : Q \times \Sigma \to Q$$

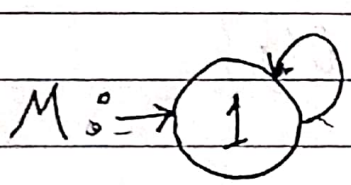
* Extended Transition Function δ^*

$$\delta^* : Q \times \Sigma^* \to Q$$

* $L(M) = \{ w, w \in \Sigma^*, \delta^*(q_0, w) \in F \}$

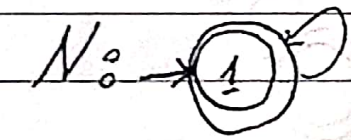
Language accepted by Machine M.

Ex1



$L(M) = \{\}$

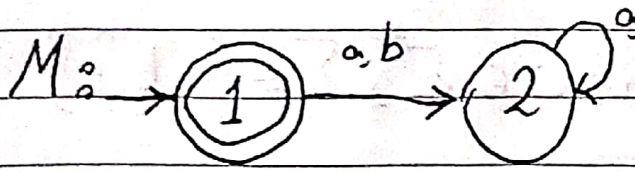
Ex 2



$L(N) = \{ w, w \in \Sigma^* \}, \Sigma = \{a, b\}$

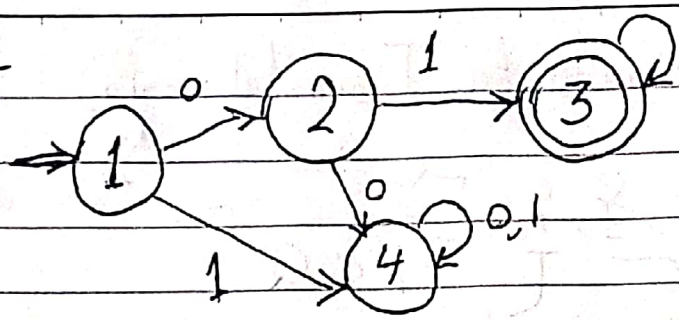
$L(N) = \Sigma^* = \{a, b\}^*$

Ex 3



$L(M) = \{\lambda\}$

Ex.4



$\Sigma = \{0,1\}$

$F = \{3\}$

$q_0 = 1, Q = \{1,2,3,4\}$

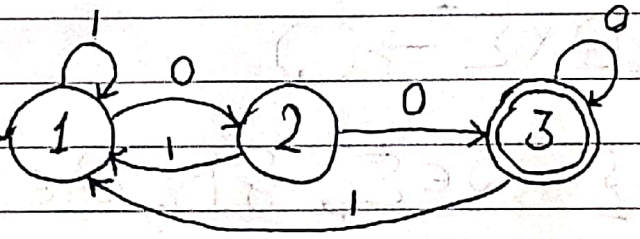
$L(M) = \{01, 010, 011, \dots\}$

$= \{01w : w \in \Sigma^*, \Sigma = \{0,1\}\}$

$= \{01\{0,1\}^*\}$

Ex.5

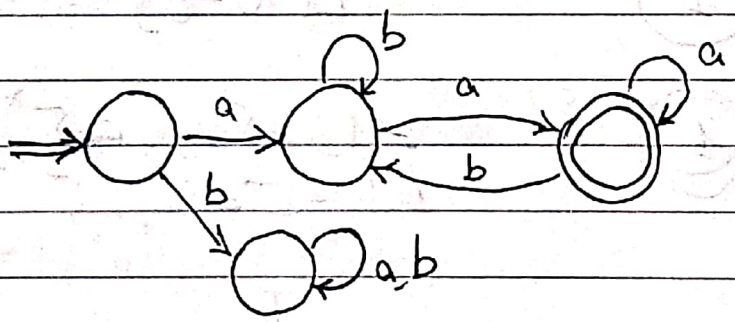
M:



$L(M) = \{00, 000, 1000, 0000, 0100, 1000, 1100, \dots\}$

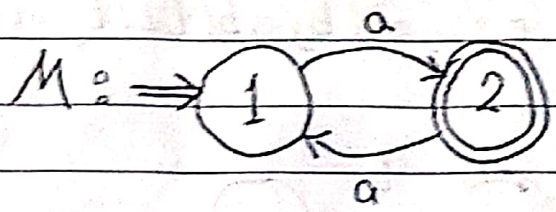
$= \{w00 : w \in \Sigma^*, \Sigma = \{0,1\}\}$

Ex.6



$L(M) = \{awa : w \in \Sigma^*, \Sigma = \{a,b\}\}$

Ex 7

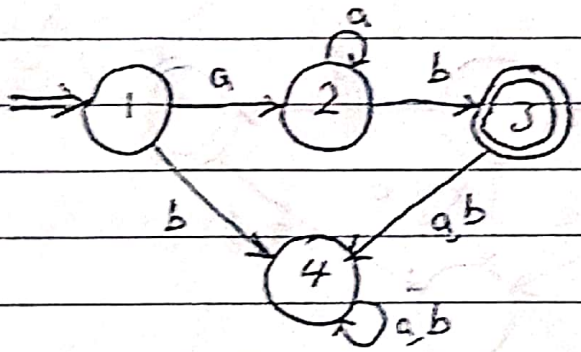


$L(M) = \{ w : w \in \{a\}^*, |w| \text{ is odd} \}$

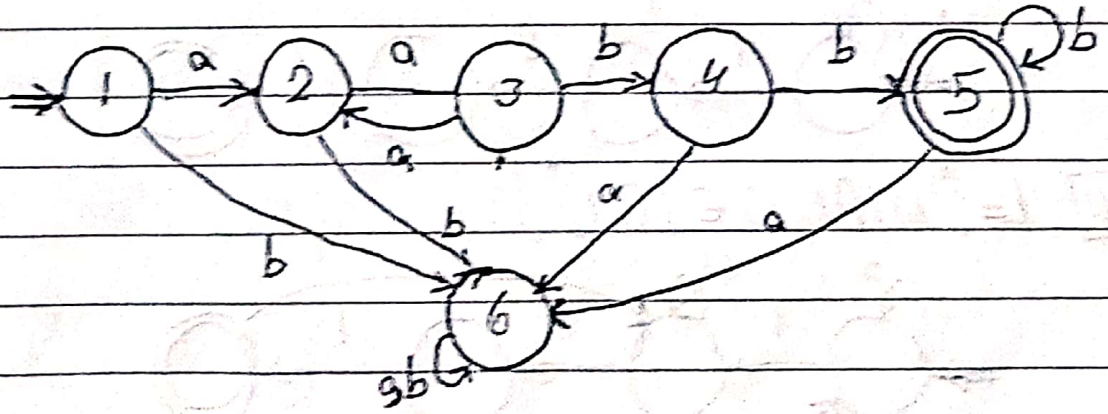
$|w| \bmod 2 = 1$

Design DFA for the following languages

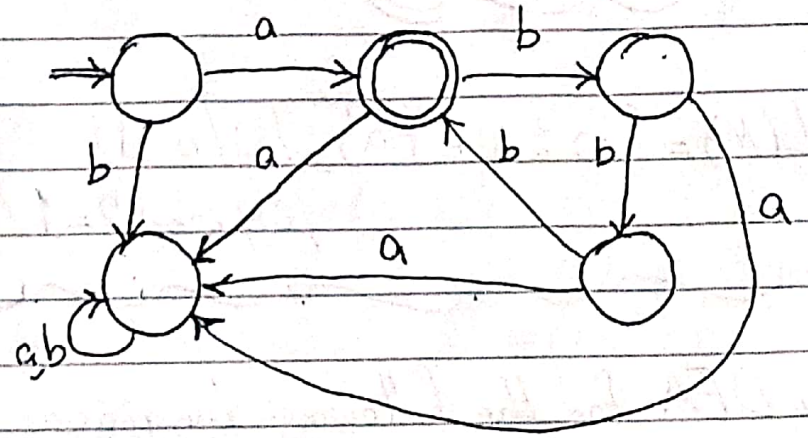
6 $L = \{ a^n b : n \geq 1 \} = \{ab, aab, aaab$



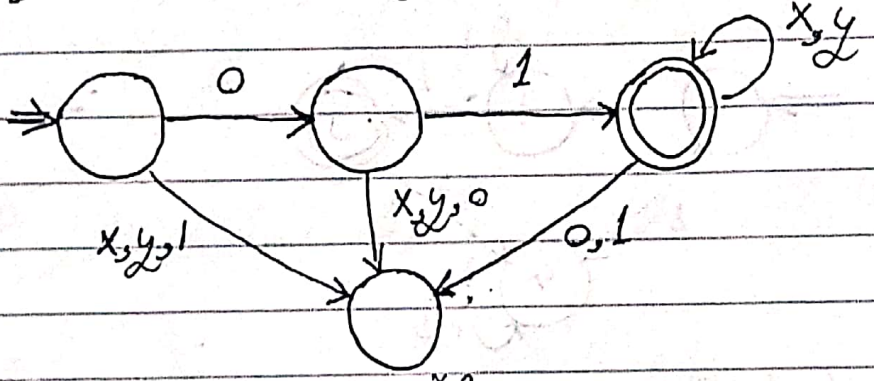
7 $L = \{ a^{2n} b^m : n \geq 1, m \geq 2 \} = \{aabb, aabbb, \ldots, aaaabb, \ldots\}$



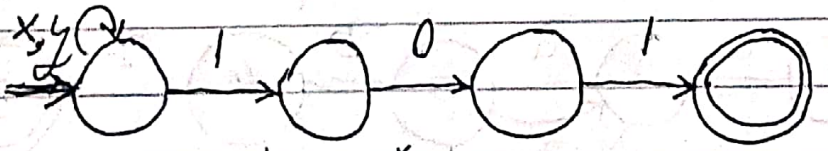
8. $L=\{ab^{3n} : n \geq 0\} = \{a, abbb, abbbbbb, \ldots\}$



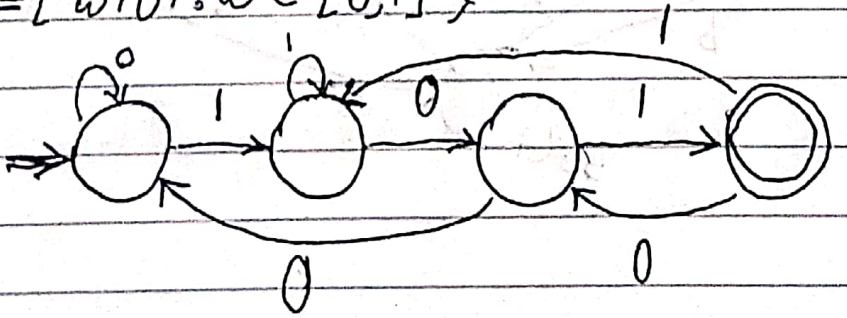
9. $L=\{0w : w \in \{x,y\}^*\}$



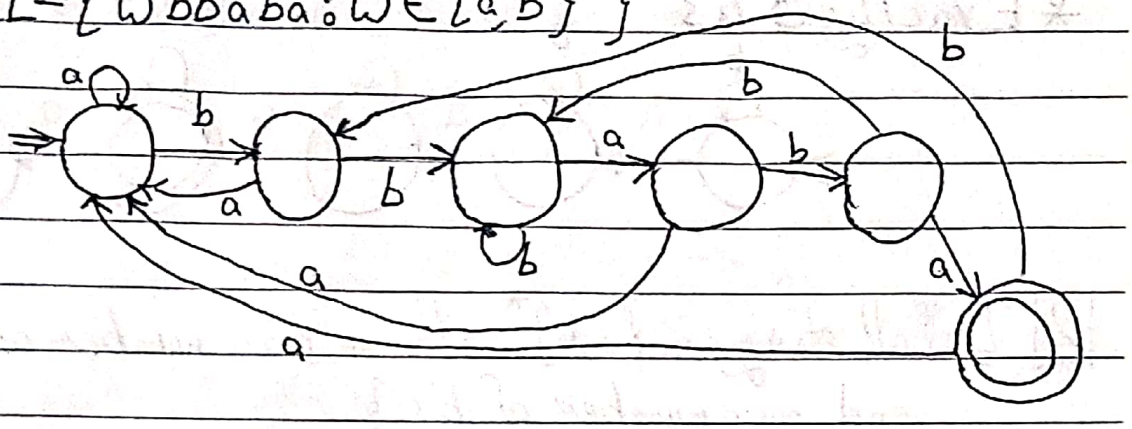
10. $L=\{w101 : w \in \{x,y\}^*\}$



11. $L=\{w101 : w \in \{0,1\}^*\}$



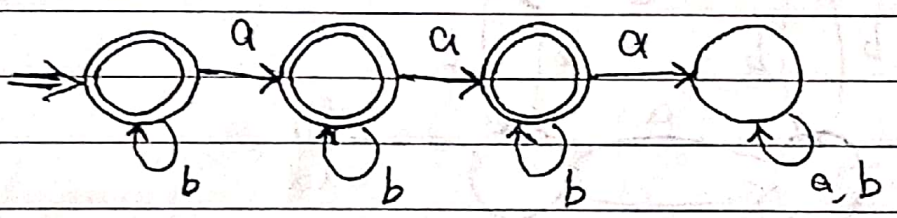
12 $L=\{wbbaba : w \in \{a,b\}^*\}$



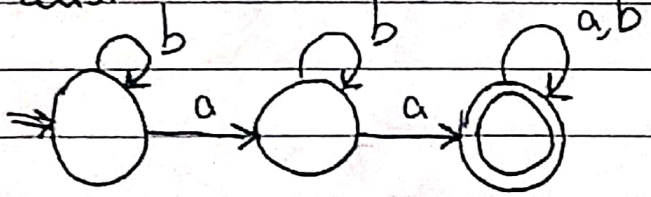
* Design DFA :-

11 $L=\{$all strings in $\{a,b\}^*$ that contain at most 2 a's$\}$

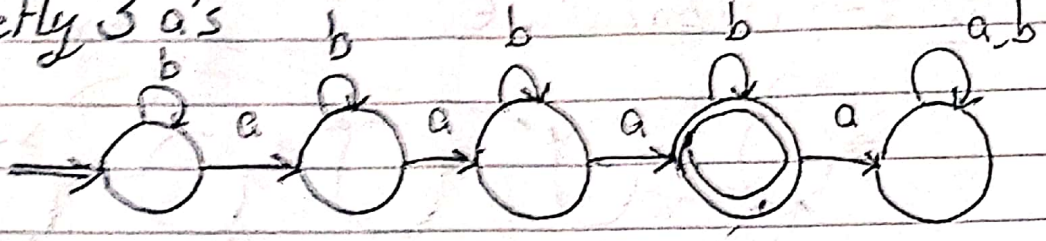
$=\{\lambda, a, aa, bbb, \ldots, ab, ba, abb, bba, bab, bbab, babb, \ldots, baa, aba, aab, \ldots\}$



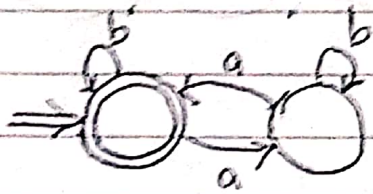
* at least



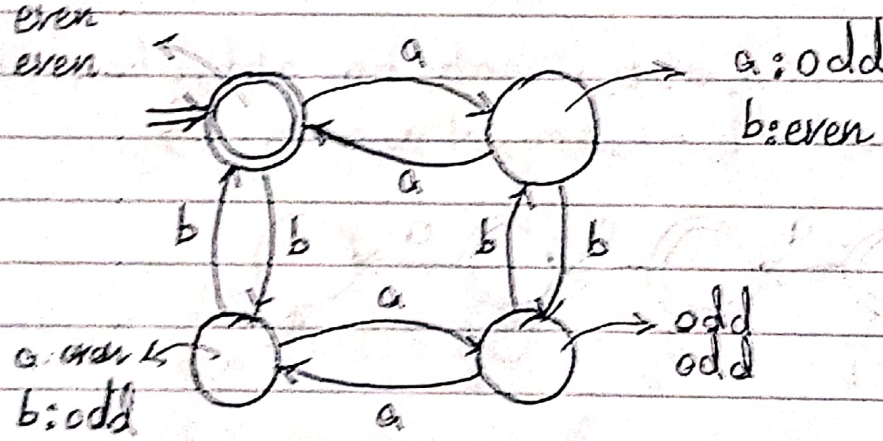
* Exactly 3 a's



2 L={all strings in {a,b}* with an even number of a's and any number of b's}



* even of a's and even of b's



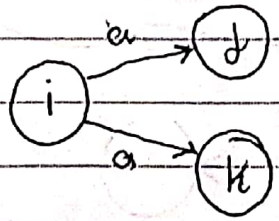
* NFA : Non-Deterministic Finite Automata

$NFA = (Q, \Sigma, \delta, q_0, F)$

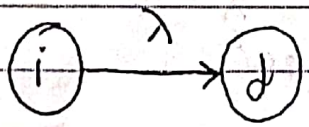
$\delta : Q \times \Sigma \cup \{\lambda\} \to \underbrace{2^Q}_{\text{Powerset}}$

assume $\Sigma = \{a, b\}$

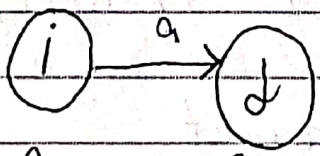
Case#1



Case#2

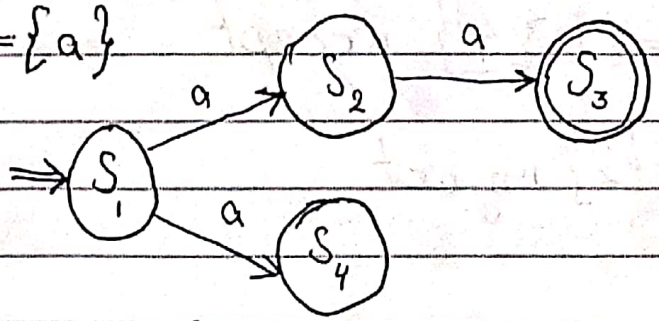


Case#3



transition function for 'b' is undefined

Ex. $\Sigma = \{a\}$

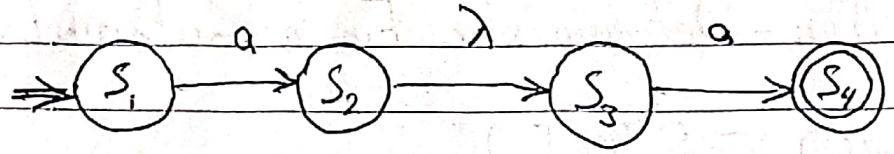


input	output
a	$\{S_2, S_4\}$ reject
aa	$\{S_3\}$ accept
aaa	$\emptyset$ reject

* $\{S_1\} \xrightarrow{a} \{S_2, S_4\} \cap F = \emptyset$
* $\{S_1\} \xrightarrow{a} \{S_2, S_4\} \xrightarrow{a} \{S_3\} \cap F \neq \emptyset$

1. $\delta^*(S_1, a) = \{S_2, S_4\}$
2. $\delta^*(S_1, aa) = \{S_3\} \in F$
3. $\delta^*(S_1, aaa) = \emptyset$ reject

Ex

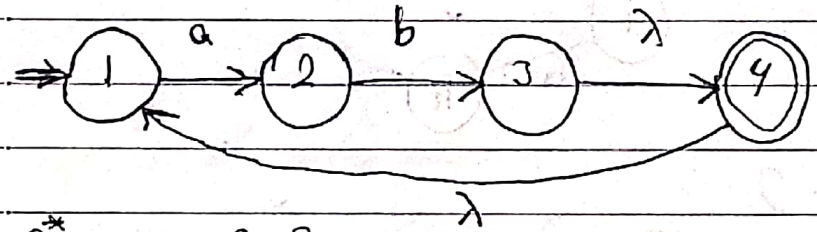


[1] $\hat{\delta}^*(S_1, a) = \{S_2, S_3\}$

[2] $\hat{\delta}^*(S_1, aa) = \{S_4\}$ accept ← tracing

$\{S_1\} \xrightarrow{a} \{S_2, S_3\} \xrightarrow{a} \{S_4\} \in F$

Ex.



[1] $\hat{\delta}^*(1, a) = \{2\}$ reject

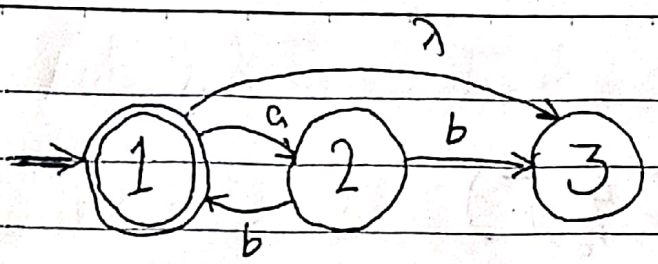
[2] $\hat{\delta}^*(1, ab) = \{3, 4, 1\} \cap F$ accept

[3] $\hat{\delta}^*(1, aa) = \emptyset$ reject

[4] $\hat{\delta}^*(1, abab) = \{1, 3, 4\}$ accept

[5] $\hat{\delta}^*(1, aba) = \{2\}$ reject

Ex

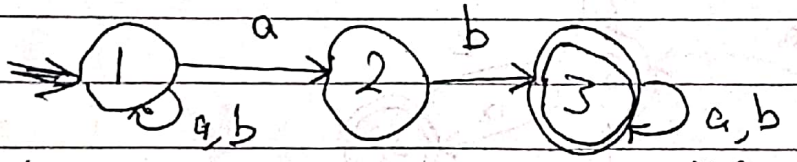


input	output
λ	$\delta^*(1,\lambda)=\{1,3\}$ accept
ab	$\delta^*(1,ab)=\{1,3\}$ accept

$L(M)=\{\lambda, ab, abab, ababab, \ldots\}$

$=\{(ab)^n : n \geq 0\}$

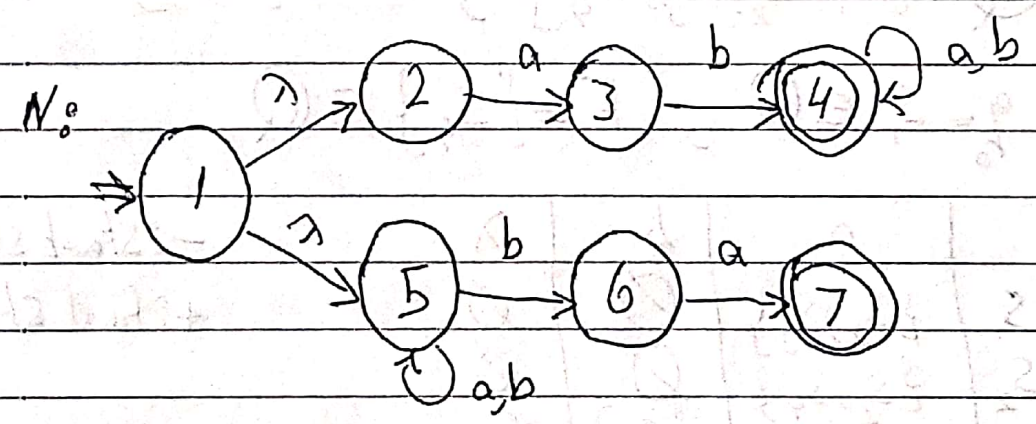
Ex



$L(M)=\{w_1 ab w_2 : w_1, w_2 \in \{a,b\}^*\}$

$=\{$ all strings in $\{a,b\}^*$ containing substring "ab" $\}$

Ex.



$L(N)=\{w_1 ba \cup ab w_2 : w_1, w_2 \in \{a,b\}^*\}$

$=\{wba : w \in \{a,b\}^*\} \cup \{abw : w \in \{a,b\}^*\}$

Formal Def.

$$N = (Q, \Sigma, \delta, q_0, F)$$

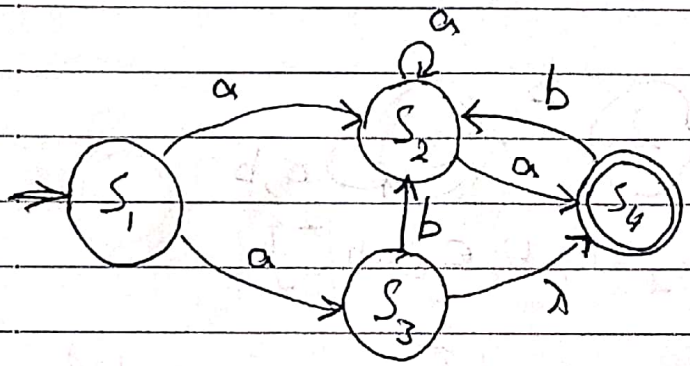
(transition function) $\delta : Q \times \Sigma \cup \{\lambda\} \to 2^Q$

power set

$\lambda \notin \Sigma$

by def $\delta^*(q, \lambda) = q$

Ex



$Q = \{S_1, S_2, S_3, S_4\}$, $\Sigma = \{a, b\}$

$q_0 = S_1 \in Q$, $F = \{S_4\} \subseteq Q$

	a	b	λ
− S_1	$\{S_2, S_3\}$	$\emptyset$	$\emptyset$
S_2	$\{S_2, S_4\}$	$\emptyset$	$\emptyset$
S_3	$\emptyset$	$\{S_2\}$	$\{S_4\}$
+ S_4	$\emptyset$	$\{S_2\}$	$\emptyset$

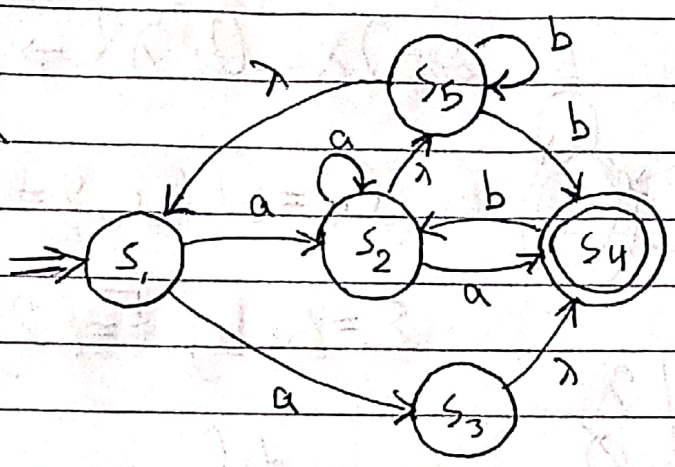
−: Start state

+: final state

* Extended Transition Function

$$\delta^*(q, \omega) = q' \quad \tau \in \Sigma$$

Ex



Find

$\delta^*(S_1, \lambda) = \{S_1\}$

$\delta^*(S_3, \lambda) = \{S_3, S_4\}$

$\delta^*(S_2, \lambda) = \{S_2, S_5, S_1\}$

$\delta^*(\{S_1, S_2\}, \lambda) = \{S_1, S_2, S_5\}$

$\delta^*(\{S_3, S_4\}, \lambda) = \{S_3, S_4\}$

⊛ $\delta^*(S_1, a) = \{S_1, S_2, S_3, S_4, S_5\}$

$\{S_1\} \xrightarrow{a} \{S_2, S_3, S_5, S_1, S_4\}$

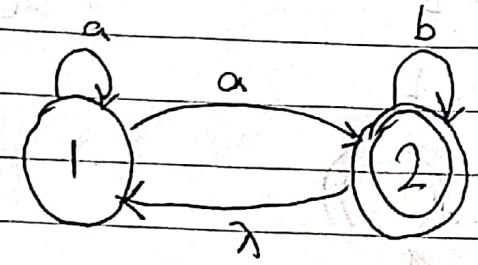
⊛ $\delta^*(S_1, aa) = \{S_1, S_2, S_3, S_4, S_5\}$

⊛ $\delta^*(S_1, ab) = \{S_1, S_2, S_4, S_5\}$

⊛ NFA $N = (Q, \Sigma, \delta, q_0, F)$

$\delta : Q \times \Sigma \cup \{\lambda\} \to 2^Q$

Ex.



$Q = \{1, 2\}$, $q_0 = 1$

$\Sigma = \{a, b\}$, $F = \{2\}$

	a	b	λ
1	{1, 2}	∅	∅
2	∅	{2}	{1}

$\delta(1, a) = \{1, 2\}$	$\delta(2, a) = \{\}$
$\delta(1, b) = \{\}$	$\delta(2, b) = \{2\}$
$\delta(1, \lambda) = \{\}$	$\delta(2, \lambda) = \{1\}$

def

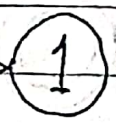
1) $\delta^*(q, \lambda) = q$ + any reacable state(s) using 'λ'

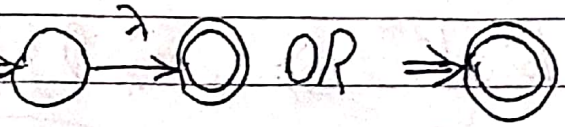
2) $\delta^*(S) = S$ + any reacable state(s) using 'λ'

$\delta^* : \delta^*(q, w) \to 2^Q$, $w \in \Sigma^*$

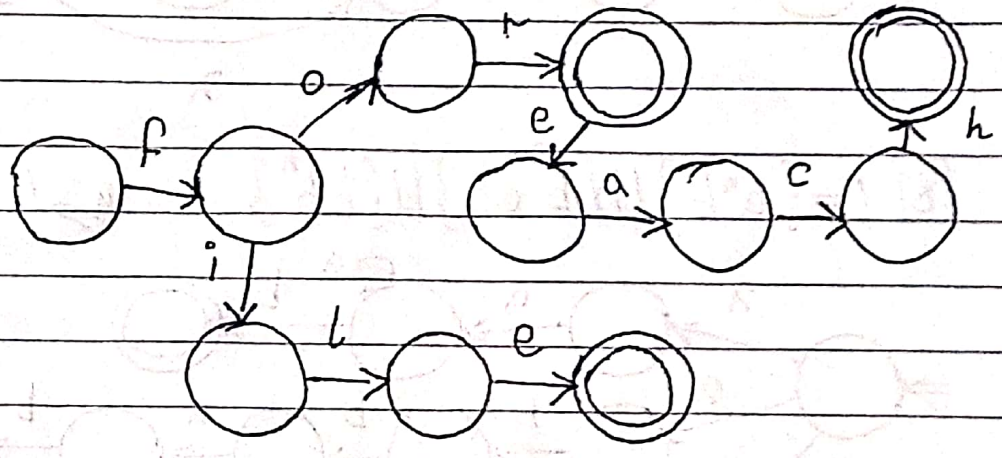
1. $\delta^*(1, aa) = \{1, 2\}$ accept ($\in F$)
2. $\delta^*(1) = \{1\}$
3. $\delta^*(2) = \delta^*(2, \lambda) = \{1, 2\}$ accept
4. $\delta^*(1, abb) = \{1, 2\}$ accept
5. $\delta^*(1, aba) = \{1, 2\}$ accept

* Design NFA for the following languages :-

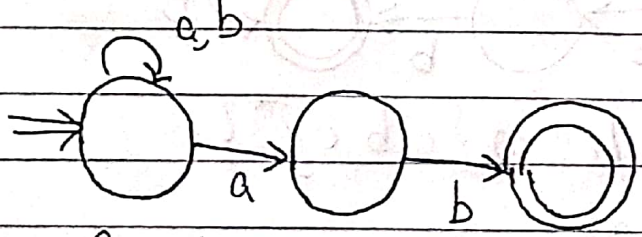
[1] $L = \{\}$ 

[2] $L = \{\lambda\}$ 

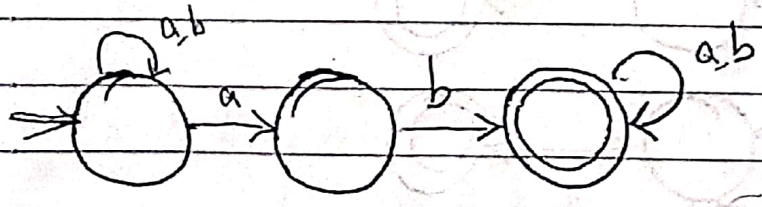
[3] $L = \{for, file, foreach\}$



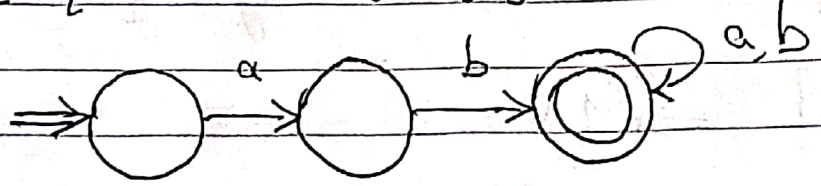
[4] $L = \{wab, w \in \{a,b\}^*\}$



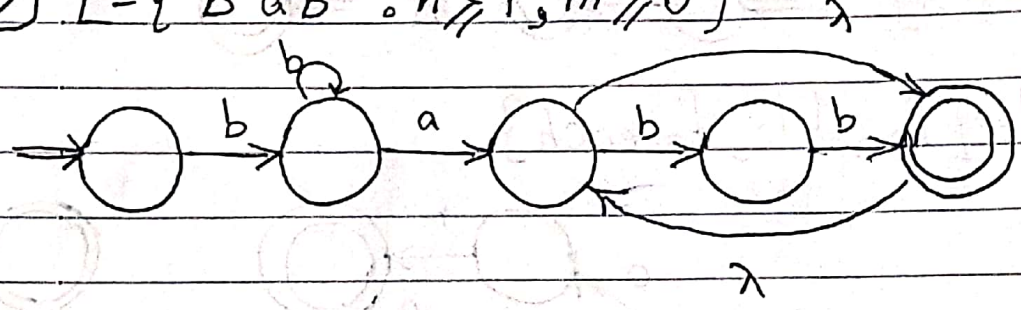
[5] $L = \{w_1 ba w_2 \ni w_1, w_2 \in \{a,b\}^*\}$



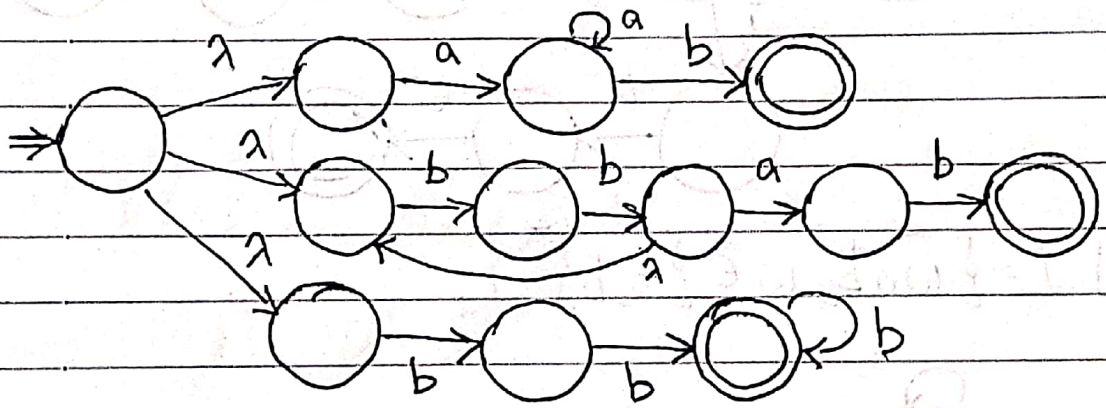
[6] $L = \{ abw : w \in \{a,b\}^* \}$



[7] $L = \{ b^n a b^{2m} : n \geq 1, m \geq 0 \}$

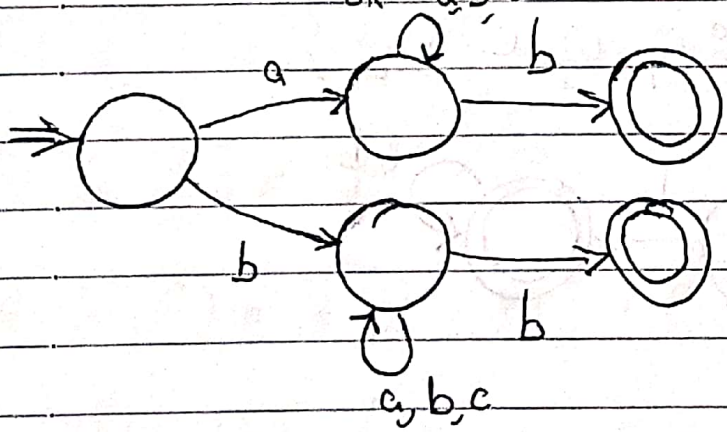


[8] $L = \{a^n b\} \cup \{b^{2n} ab\} \cup \{bb^n\}$ $n \geq 1$



[9] $L = \{ awa \mid bwb : w \{a,b,c\}^* \}$

↓ OR a,b,c



* Algorithm to convert NFA to DFA:

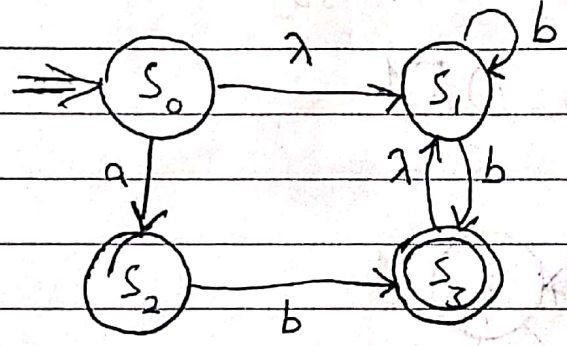
Algorithm name: subset construction Algorithm

Input: NFA $N = (S, \Sigma, \delta_{NFA}, S_0, F_{NFA})$

Output: DFA $M = (Q, \Sigma, \delta_{DFA}, q_0, F_{DFA})$

Ex.

NFA N:



* Step 1: Compute q_0 (DFA Start State)

$$q_0 = \delta^*(S_0, \lambda) = \{S_0, S_1\}$$

* Step 2: Construct δ_{DFA} table

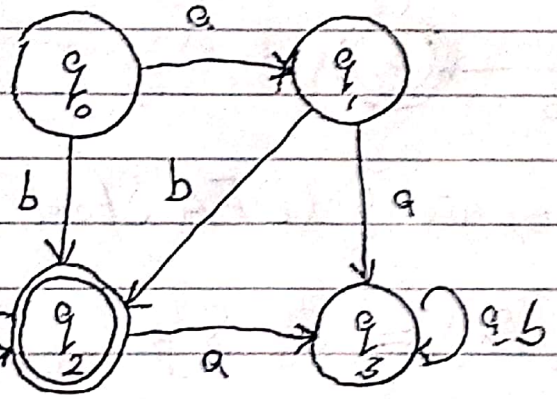
	a	b
q_0 $\{S_0, S_1\}$	$\delta^*(\{S_0, S_1\}, a)$ $\{S_2\}$	$\delta^*(\{S_0, S_1\}, b)$ $\{S_1, S_3\}$
q_1 $\{S_2\}$	$\delta^*(\{S_2\}, a)$ $\emptyset$	$\delta^*(\{S_2\}, b)$ $\{S_1, S_3\}$

		a	b
q_2	$\{s_1, s_3\}$	$\delta^*(\{s_1, s_3\}, a)$ $\emptyset$	$\delta^*(\{s_1, s_3\}, b)$ $\{s_1, s_3\}$
q_3	$\emptyset$	$\delta^*(\emptyset, a) = \emptyset$	$\delta^*(\emptyset, b) = \emptyset$

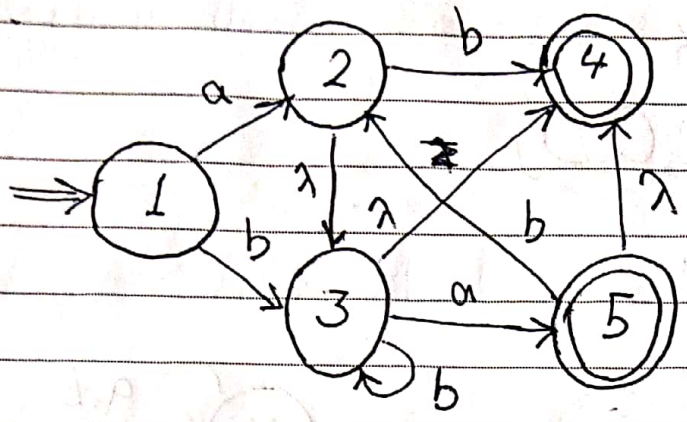
* Step 3: Compute F_{DFA}

$$F_{DFA} = \{q_2\} = \{\{s_1, s_3\}\}$$

* Step 4: Draw DFA



Ex.



1] Compute DFA Start State

$$A = \delta^*(1, \lambda) \Rightarrow A = \{1\}$$

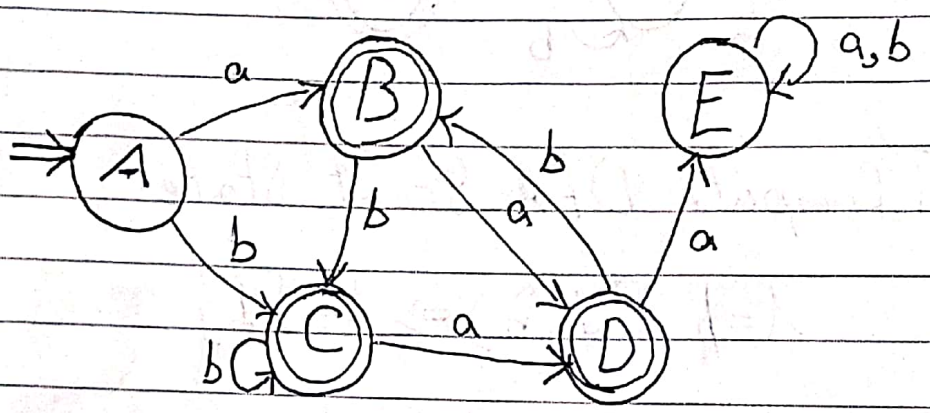
2] Construct δ_{DFA} table

δ_{DFA}	a	b
A	$\delta^*(A,a)=\{2,3,4\}$ B	$\delta^*(A,b)=\{3,4\}$ C
B {2,3,4}	$\delta^*(B,a)=\{4,5\}$ D	$\delta^*(B,b)=\{3,4\}$ C
C {3,4}	$\delta^*(C,a)=\{4,5\}$ D	$\delta^*(C,b)=\{3,4\}$ C
D {4,5}	$\delta^*(D,a)=\{\}$ E	$\delta^*(D,b)=\{2,3,4\}$ B
E {}	E	E

[3] Compute F_{DFA}

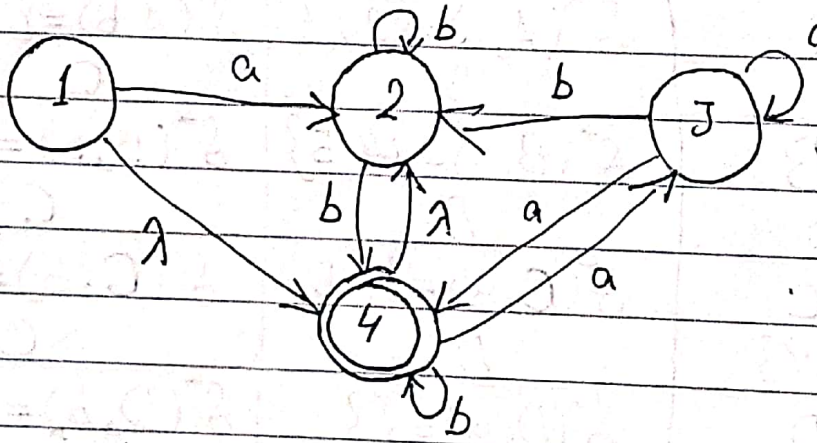
$F_{DFA} = \{B, C, D\}$

[4] Draw DFA



* Convert NFA to DFA :

Ex



[1] Compute DFA start state

$A = \delta^*(1) = \{1, 2, 4\}$

Note :

$\delta^*(q) = \delta^*(q, \lambda)$

[2] Compute DFA transition table (δ_{DFA}) -: Start State
+: Final State(s)

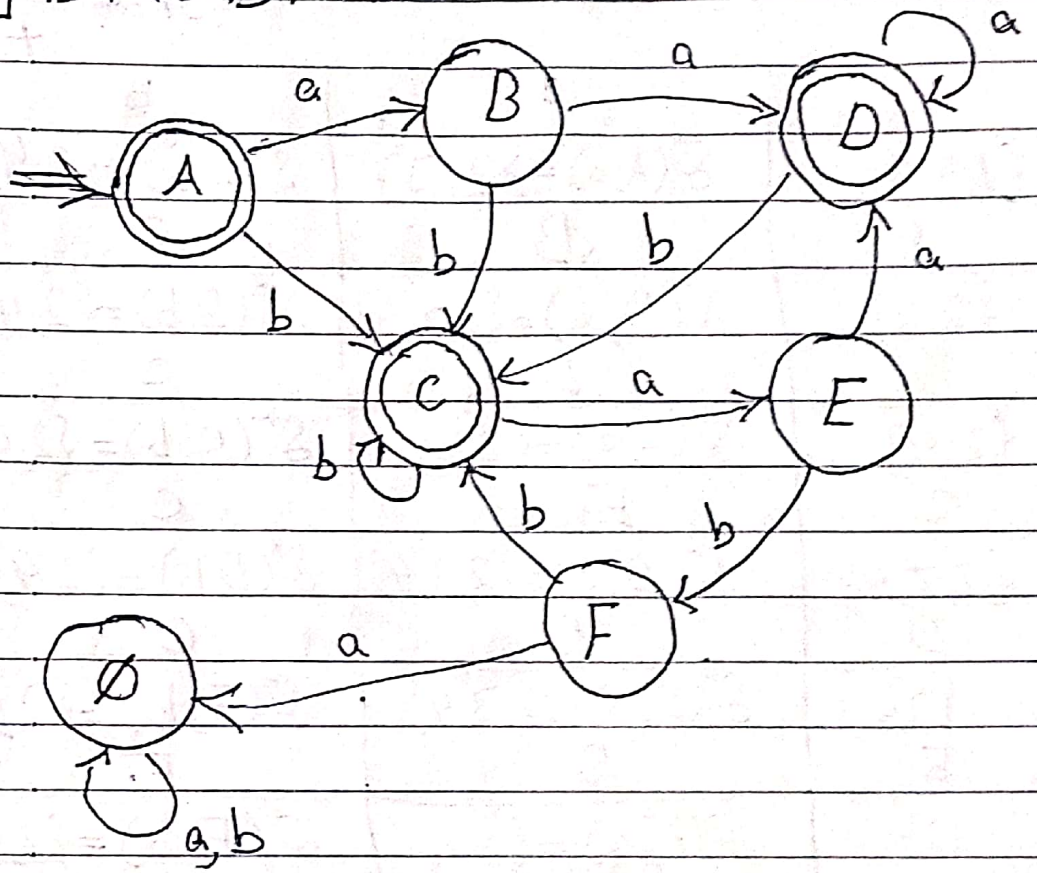
	a	b
- + {1,2,4} A	$\delta^*(A,a)=\{2,3\}$ B	$\delta^*(A,b)=\{2,4\}$ C
{2,3} B	$\delta^*(B,a)=\{2,3,4\}$ D	$\delta^*(B,b)=\{2,4\}$ C
+ {2,4} C	$\delta^*(C,a)=\{3\}$ E	$\delta^*(C,b)=\{2,4\}$ C
+ {2,3,4} D	$\delta^*(D,a)=\{2,3,4\}$ D	$\delta^*(D,b)=\{2,4\}$ C
{3} E	$\delta^*(E,a)=\{2,3,4\}$ D	$\delta^*(E,b)=\{2\}$ F
{2} F	$\delta^*(F,a)=\{\}$ ∅	$\delta^*(F,b)=\{2,4\}$ C
∅	∅	∅

~~[illegible]~~

[3] Compute final state(s)

$$F_{DFA}=\{A,C,D\}=\{\{1,2,4\},\{2,4\},\{2,3,4\}\}$$

[4] Draw DFA

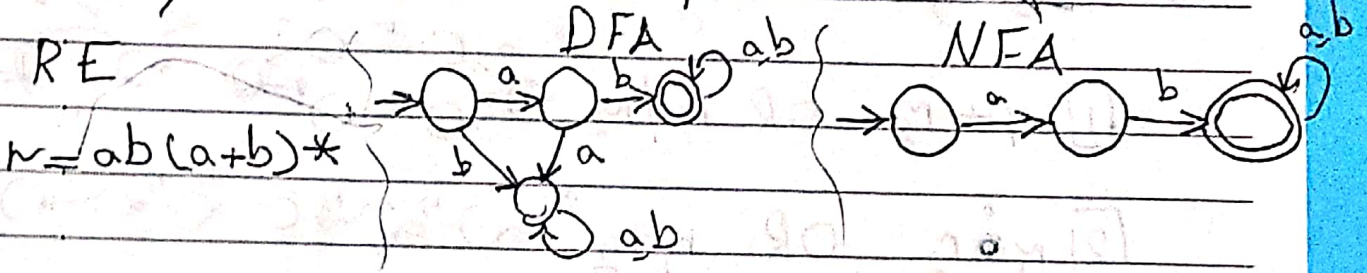


* Regular Expressions

describe Regular Languages.

Regular languages

* $L = \{abw : w \in \{a,b\}\}$,

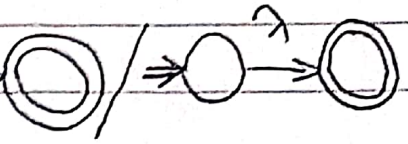


* $R = (a + b \cdot c)^*$ → Kleene Closure
  * : 0, 1, 2, .., ∞
  + : OR
  · : Concatenation

$L(R) = \{a, bc\}^* = \{\lambda, a, bc, aa, abc, bca, bcbc, \ldots\}$

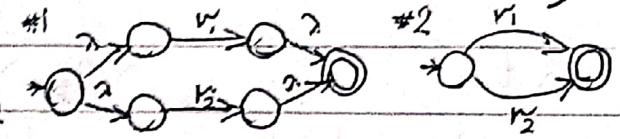
Primitive REs:

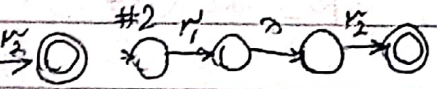
1. $r = \emptyset$ $\quad L(\emptyset) = \{\}$

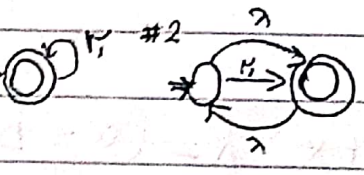
2. $r = \lambda$ $\quad L(\lambda) = \{\lambda\}$ 

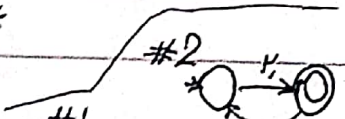
3. $r = a$ $\quad a \in \Sigma, L(a) = \{a\}$

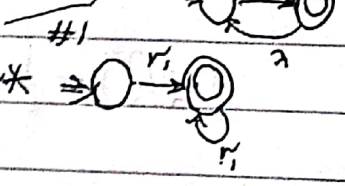
* REs operations (given r_1 and r_2 are REs)

1. $r_1 + r_2$ OR $r_1 | r_2$ 

2. $r_1 \bullet r_2$ OR $r_1 r_2$ 

3. r_1^* $\quad (* : 0, 1, 2, \ldots, \infty)$ 

4. (r_1) $\quad a + b^* \neq (a+b)^*$ 

5. r_1^+ $\quad (+ : 1, 2, 3, \ldots, \infty) / r_1^+ = r_1 r_1^*$ 

6. $r_1?$: $(? : \emptyset \text{ or } 1) / r_1? = r_1 + \lambda$

$r = \emptyset \qquad L(\emptyset) = \{\}$

$r = a \qquad L(a) = \{a\} \quad a \in \Sigma$

$r = \lambda \qquad L(\lambda) = \{\lambda\}$

$r_1 + r_2 \Rightarrow L(r_1 + r_2) = L(r_1) \cup L(r_2)$

$r_1 r_2 \Rightarrow L(r_1 r_2) = L(r_1) L(r_2)$

$r^* \Rightarrow L(r^*) = (L(r))^*$

$r^+ \Rightarrow L(r^+) = (L(r))^+$

$r? \Rightarrow L(r?) = L(r) \cup L(\lambda)$

Ex. Find $L(r)$

$r = (aa)^*(a+b)^* \Rightarrow L(r) = \{a^{2n} w : n \geq 0, w \{a,b\}^*\}$

$L((aa)^*(a+b)^*) \; L((aa)^* \cdot L((a+b)^*) = L(aa)^* \cdot L(a+b)^*$

$\Rightarrow [L(a) \cdot L(a)]^* \cdot [L(a) \cup L(b)]^*$

$= [\{a\} \cdot \{a\}]^* \cdot [\{a,b\}]^* \Rightarrow \{aa\}^* \cdot \{a,b\}^*$

$\Rightarrow \{\lambda, aa, aaaa, aaaaaa, \ldots\} \{\lambda, a, b, aa, ab, ba, bb, \ldots\}$

$\Rightarrow \{\lambda, a, b, aa, ab, \ldots, aaa, aab, \ldots\} \Rightarrow \{a^{2n} w : n \geq 0, w \in \{a,b\}^*\}$

* Find $L(r)$ $r=(b+ab^*a)^*$

$L(r)=\{w : w\in\Sigma^*, \#\text{of } a\text{'s is even}\}$

$L((b+ab^*a)^*) \Rightarrow L(b+ab^*a)^*$

$\Rightarrow [L(b)\cup L(ab^*a)]^* \Rightarrow [\{b\}\cup L(a)L(b^*)L(a)]^*$

$\Rightarrow [\{b\}\cup\{a\}\{b\}^*\{a\}]^*$

$\Rightarrow \{b, aa, aba, abba, \ldots\}^*$

$\Rightarrow \{\lambda, b, aa, aba, abba, \ldots, baa, aab, baba, abab, \ldots\}$

* $r=(0+1)^*00(0+1)^*$

$L(r)=\{$all strings in Σ^* containing substring $00\}$

$L(r)=\{w_1 00 w_2 : w_1, w_2\in\{0,1\}^*\}$

⊛ $r=(bb)^+(aaa)^*c$

$L(r)=\{bbc, bbbbc, bbaaac, bbaaaaaac, bbaaaaaaaaaaaac, \ldots\}$

$=\{b^{2n}a^{3m}c : n\geq 1, m\geq 0\}$

* Write Regular Expression for the following languages

1. $L = \{a\}$ $r = a$
2. $L = \{$ [redacted] for, file$\}$ $r = file + for$ // $r = f(or+ile)$
3. $L = \{0,1,2,\ldots,9\}$ $r = 0+1+2+\ldots+9$ // $r = 0-9$
4. $L = \{a,b,\ldots,z,A,B,\ldots,Z\}$ $r = a+b+\ldots+z+A+B+\ldots+Z$ // $r = a-z A-Z$
5. $L = \{$integres$\}$ $r = ("+"+"-")?(0-9)^+$
6. $L = \{=, +=, -=, ++, --, -, +, ==\}$

$r = "+".("="|"+"|\lambda) | "-".("="|"-"|\lambda) | "=".("="|\lambda)$

7. $L = \{a,b\}^*$ $r = (a+b)^*$
8. $L = \{a,b\}^+$ $r = (a+b)^+$
9. $L = \{w_0 b w_1 a w_2 : w_0, w_1, w_2 \in \{0,1\}^*\}$

$r = (0+1)^*.b.(0+1)^*.a.(0+1)^*$

10. $L = \{$all strings in Σ^* containing exactly 2 a's$\}$

$r = b^*.a.b^*.a.b^*$ $\Sigma = \{a,b\}$

Write a regular expression for the following languages:-

[11] $L = \{$all strings in Σ^* containing at most 2a's, $\Sigma = \{a, b\}\}$

$$R = b^* + b^*ab^* + b^*ab^*ab^*$$

[12] at least :-

$$R = \text{[redacted]}\, b^*ab^*(ab^*)^+$$

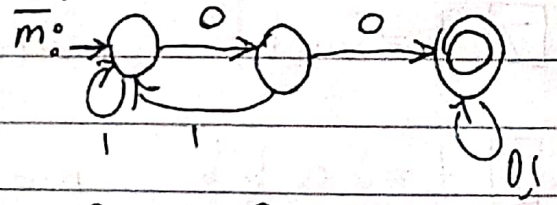
[13] at least 3a's :-

$$R = b^*ab^*ab^*ab^*(ab^*)^* \;//\; b^*ab^*ab^*(ab^*)^+$$

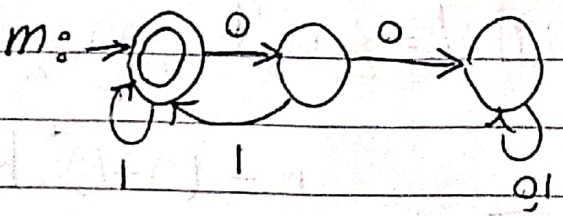
[14] $L = \{$all strings without substring 00, $\Sigma = \{0,1\}\}$

$$R = (1+01)^*(0+\lambda)$$

DFA for $\overline{L}$



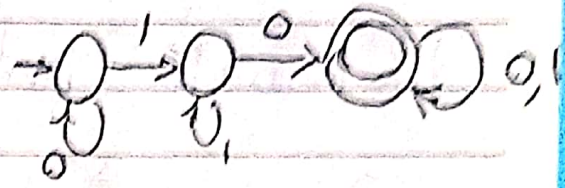
DFA for L



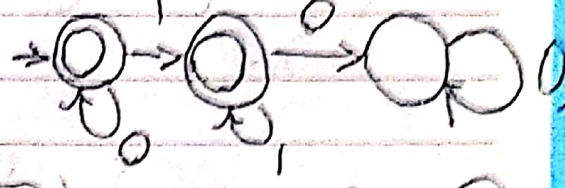
15. without substring 10!

$$R = \lambda + 0^*1^*$$

1: DFA for $\bar{L}$

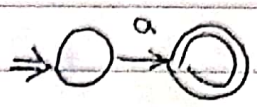


2: DFA for L

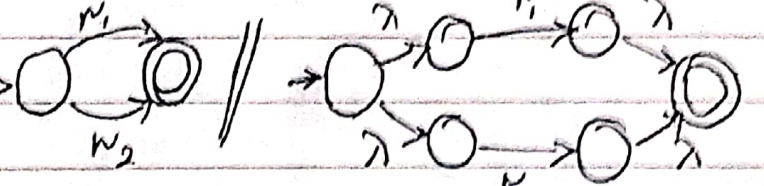


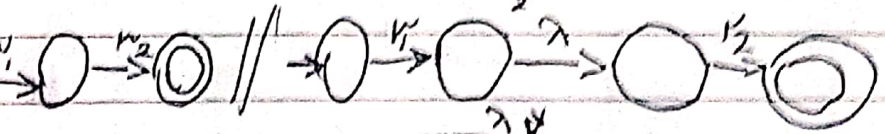
* Convert a Regular expression to NFA:

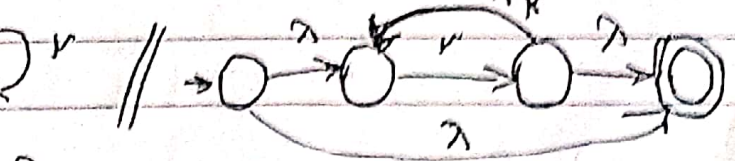
$r = \emptyset$

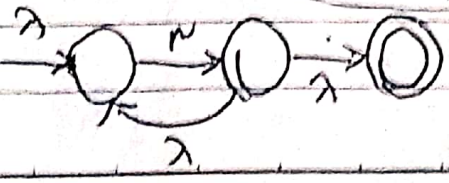
$r = a$ 

$r = \lambda$

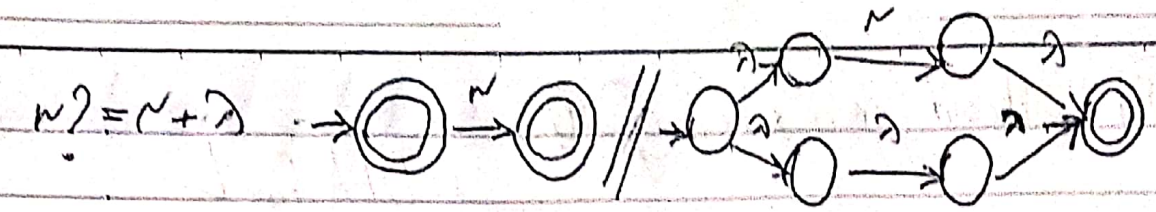
$r_1 + r_2$ 

$r_1 r_2$ 

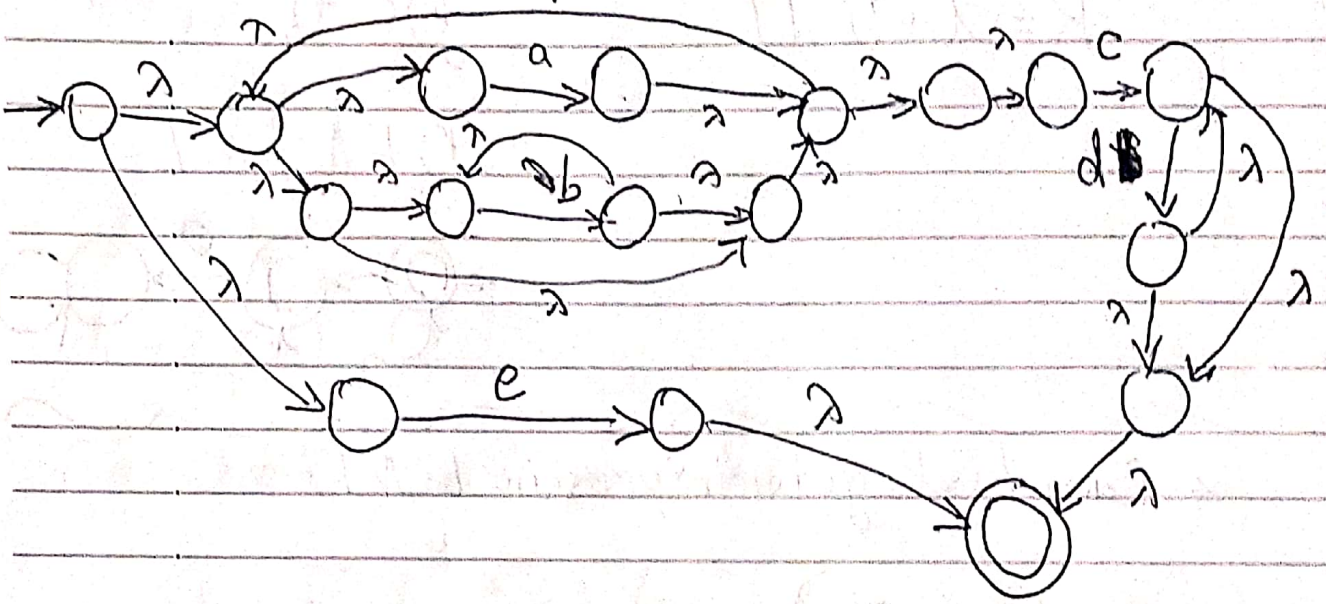
r^* 

r^+ 

$r? = r + \lambda$

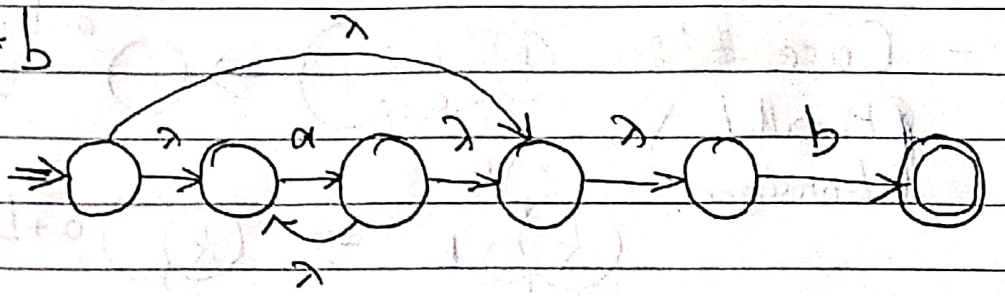


Ex $r = (a + b^*)^+ . cd^* / e$

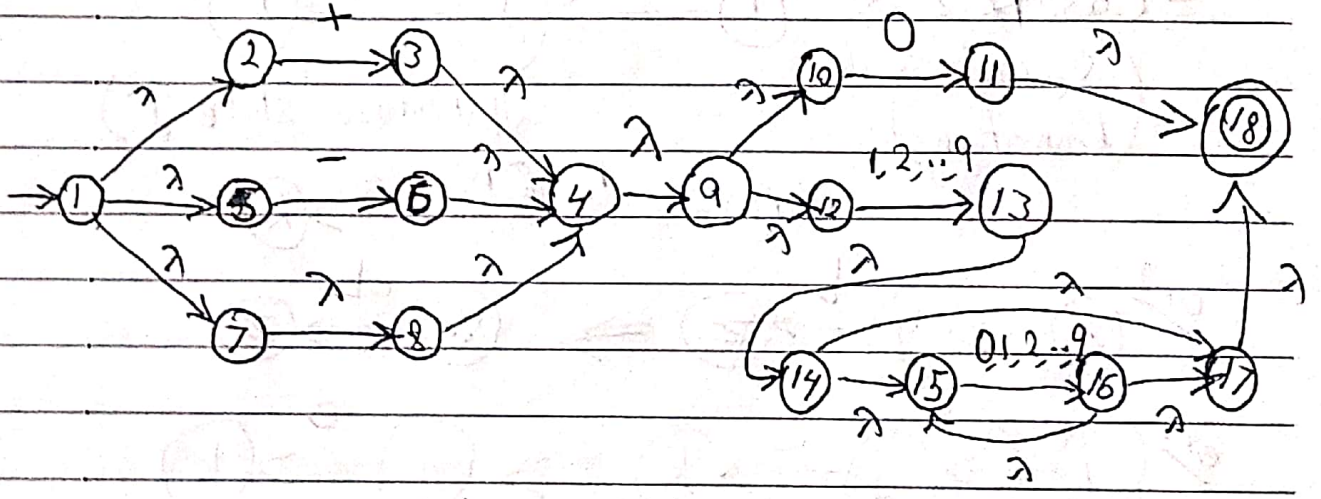


Convert RE to FA :-

[1] r = a* b



[2] r = ("+"/"−")?(0/[1-9][0-9]*)

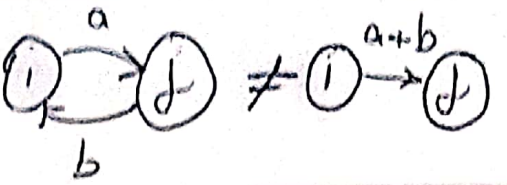


r = "−" State #: 5,6

r = "+"/"−" State #: 1, 2, 3, 4, 5, 6

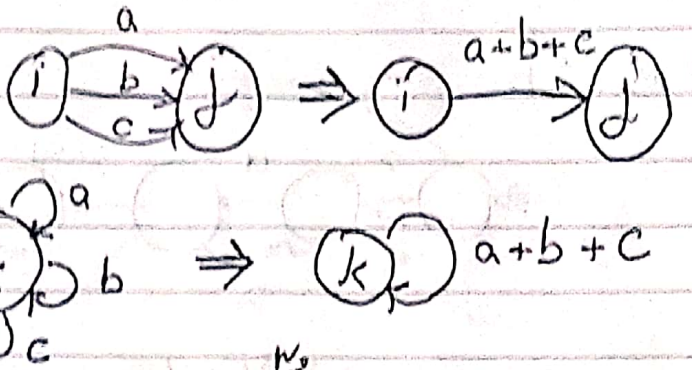
r = ("+"/"−")? State #: 1, 2, 3, 4, 5, 6, .., 8

r = (0-9)* State #: 14, 15, 16, 17

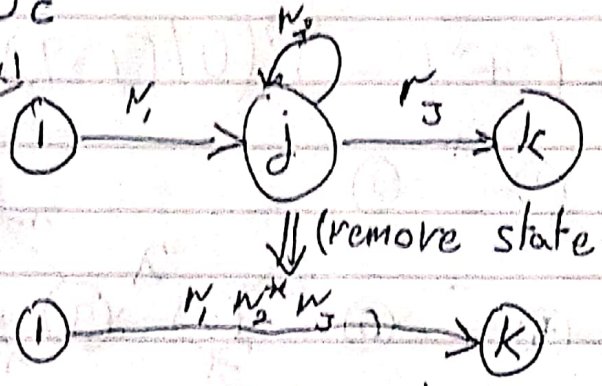
Note: 

* Convert FA to RE :-

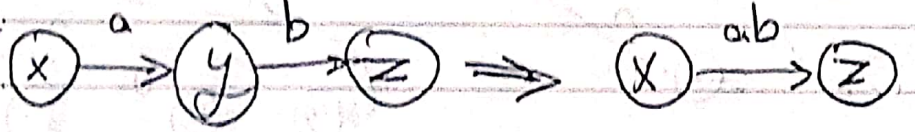
- Case #1 :- (Parallel transitions)



- Case #2 :- (Serial transitions)

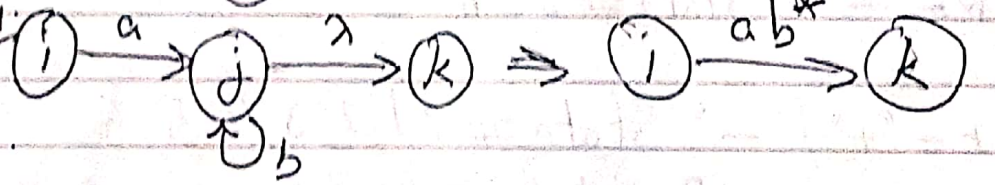


ex 2.

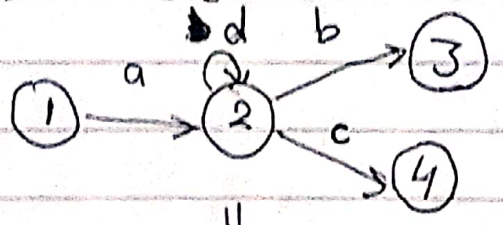


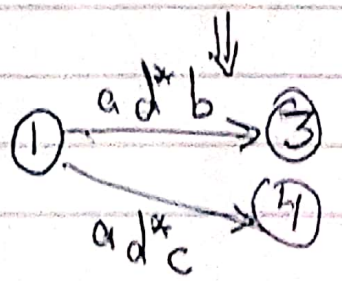
ex 3

ex 4:



- Case #3 :-



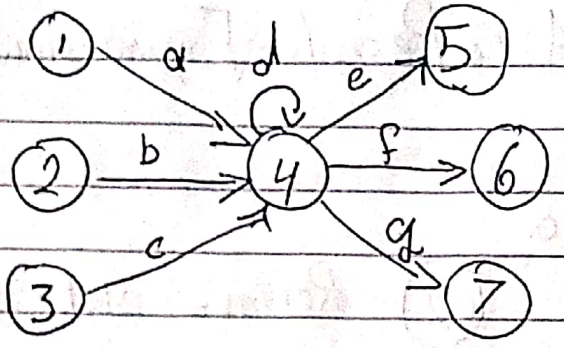


Remove (2) ✗ (without indegree loop) → outdegree

Paths 1 * 2

1 → 2 → 3 (ad*b)

1 → 2 → 4 (ad*c)



Paths ($3 \times 3 = 9$)

$1 \longrightarrow 4 \longrightarrow 5$ (ad^*e)

$1 \longrightarrow 4 \longrightarrow 6$ (ad^*f)

$1 \longrightarrow 4 \longrightarrow 7$ (ad^*g)

$2 \longrightarrow 4 \longrightarrow 5$ (bd^*e)

$2 \longrightarrow 4 \longrightarrow 6$ (bd^*f)

$2 \longrightarrow 4 \longrightarrow 7$ (bd^*g)

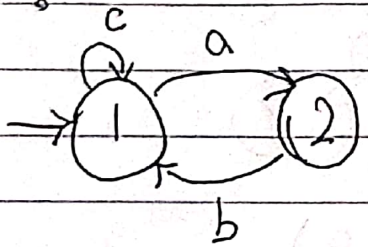
$3 \longrightarrow 4 \longrightarrow 5$ (cd^*e)

$3 \longrightarrow 4 \longrightarrow 6$ (cd^*f)

$3 \longrightarrow 4 \longrightarrow 7$ (cd^*g)

Convert NFA / DFA to Regular Expression :-

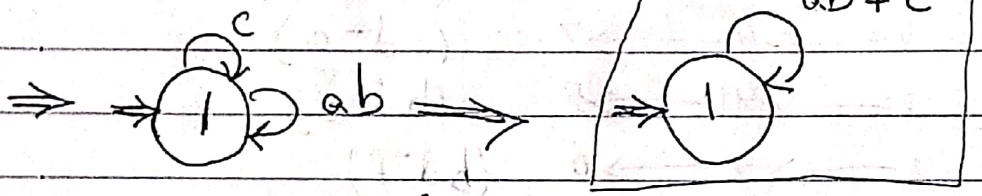
Case #4 :-



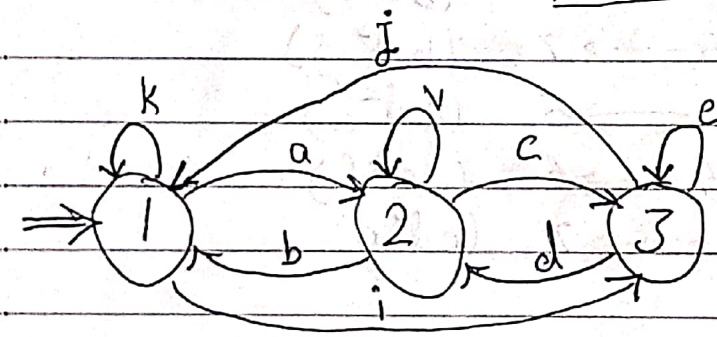
Remove state #2

Paths in = 1 Out = 1, $1 \times 1 = 1$

ex1 $1 \rightarrow 2 \rightarrow 1$ (ab)



ex2



Remove State #2

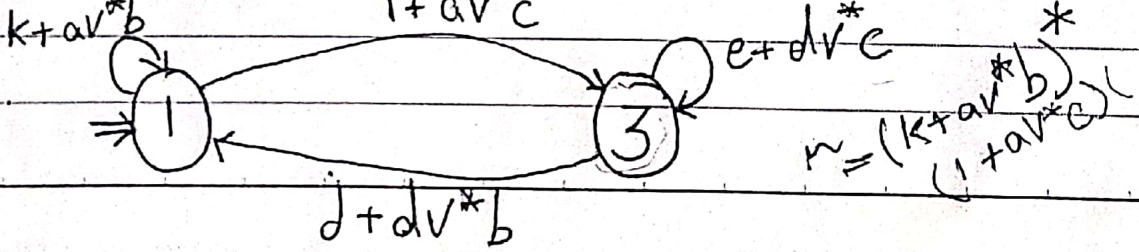
Paths in = 2, Out = 2 $2 \times 2 = 4$

$1 \rightarrow 2 \rightarrow 3$ av^*c

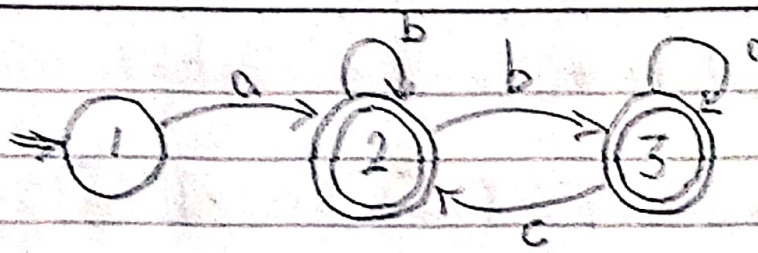
$1 \rightarrow 2 \rightarrow 1$ av^*b

$3 \rightarrow 2 \rightarrow 1$ dv^*b

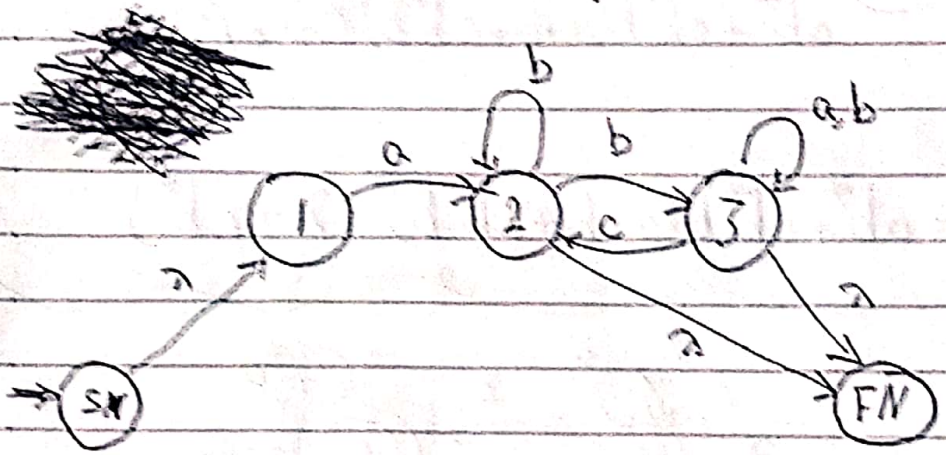
$3 \rightarrow 2 \rightarrow 3$ dv^*c



Ex.



Step 1: Create new start state (SN)
" " " accept state (FN)

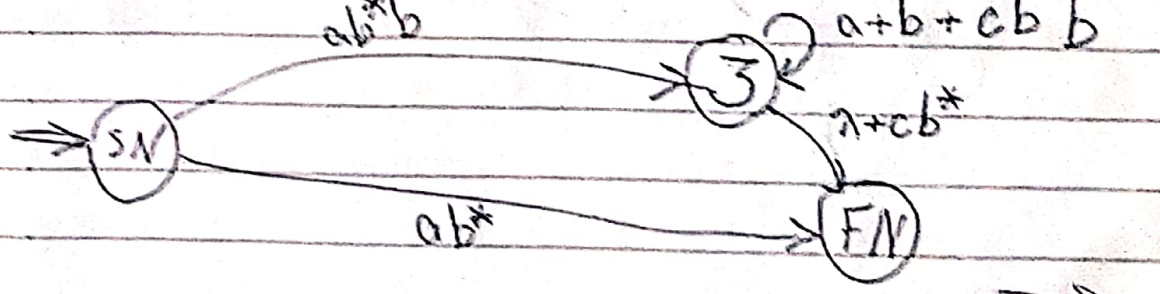


Remove state #1

SN → 1 → 2 (a)

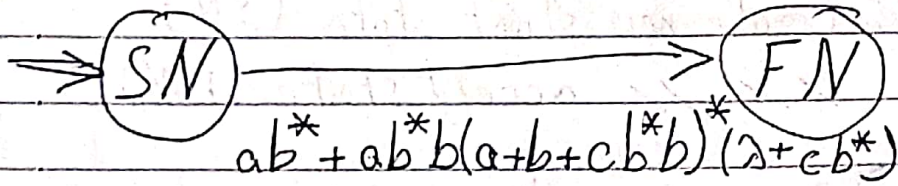
Remove state #2 in=2 out=2

Paths SN → 2 → 3 (ab^*b)
SN → 2 → FN (ab^*)
3 → 2 → 3 (cb^*b)
3 → 2 → FN (cb^*)



Remove State #3 :- in=1 out=1

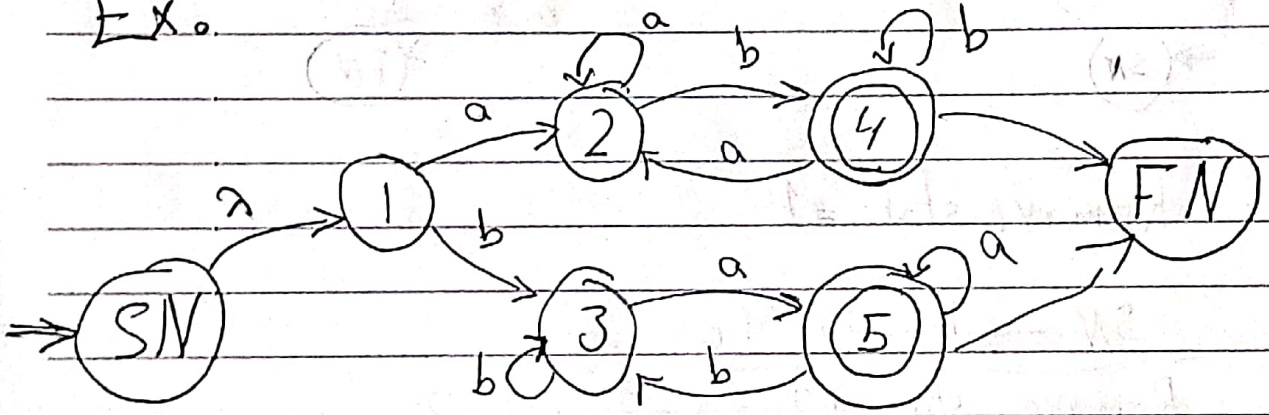
$SN \longrightarrow 3 \longrightarrow FN \quad (ab^*b(a+b+cb^*b)^*(\lambda+cb^*))$



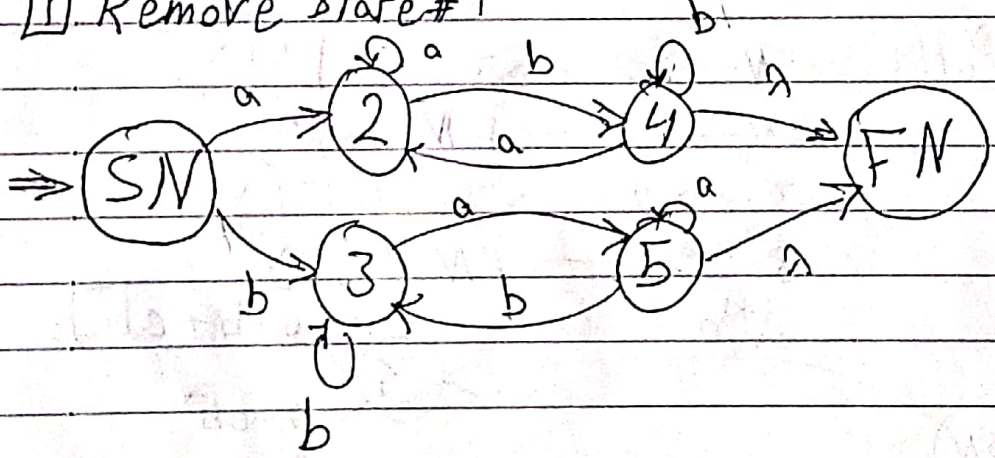
$ab^* + ab^*b(a+b+cb^*b)^*(\lambda+cb^*)$

$r = ab^* + ab^*b(a+b+cb^*b)^*(\lambda+cb^*)$

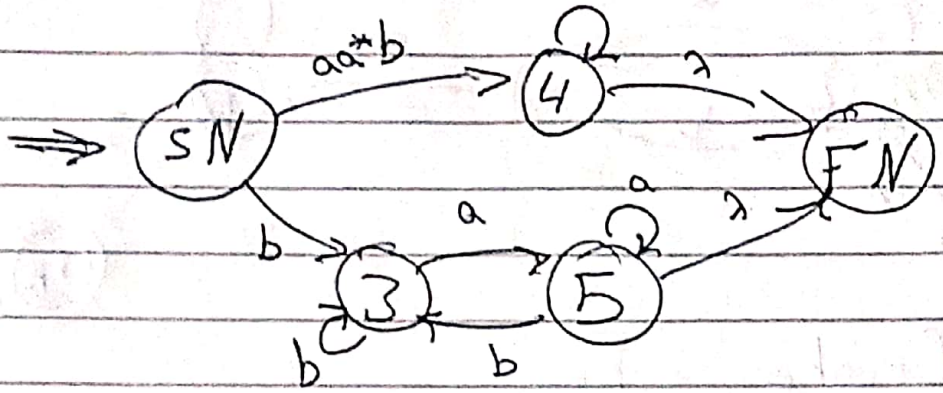
Ex.



1. Remove State #1



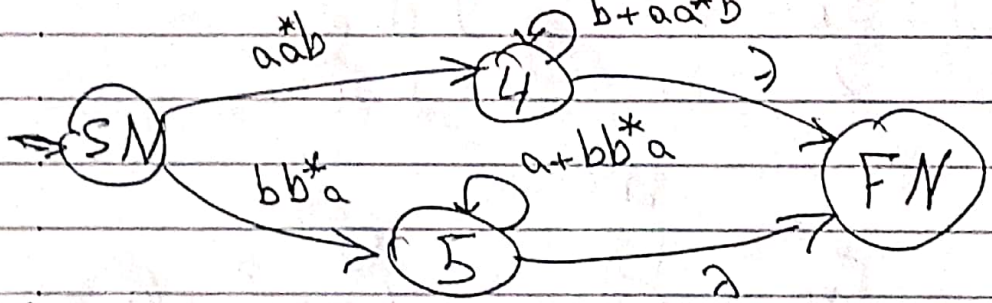
[2] Remove State #2 ~~in=2 out=1 Paths=2~~ $b+aa^*b$



[3] Remove State #3 in=2 out=1 Paths=2

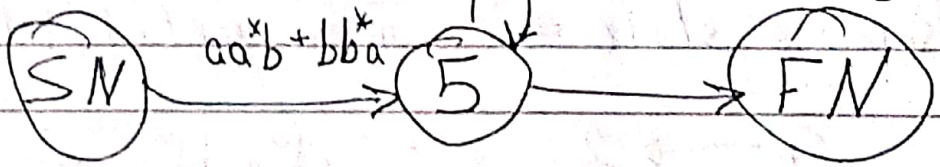
$SN \rightarrow 3 \rightarrow 5 \ (bb^*a)$

$5 \rightarrow 3 \rightarrow 5 \ (bb^*a)$



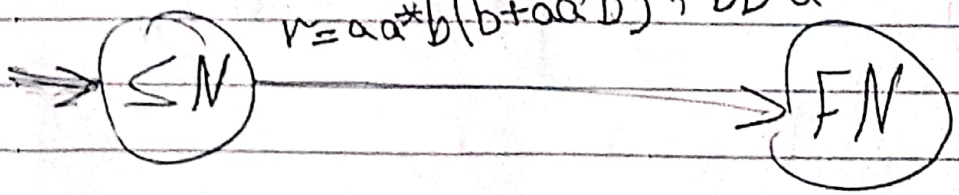
[4] Remove State #~~5~~ 4

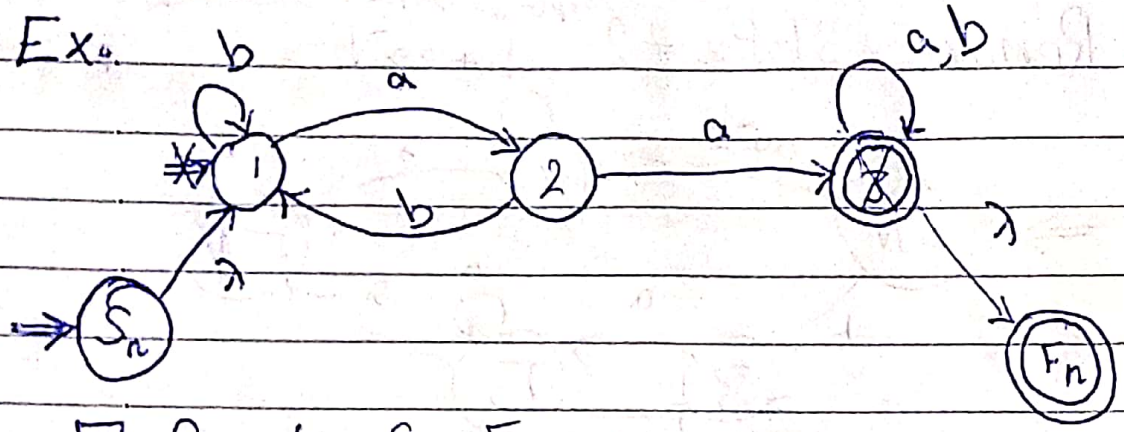
$(b+aa^*b)+(a+bb^*a)$



(5) Remove State #5

$r=aa^*b(b+aa^*b)^* + bb^*a(a+bb^*a)^*$



Ex. 

1 Create S_n, F_n

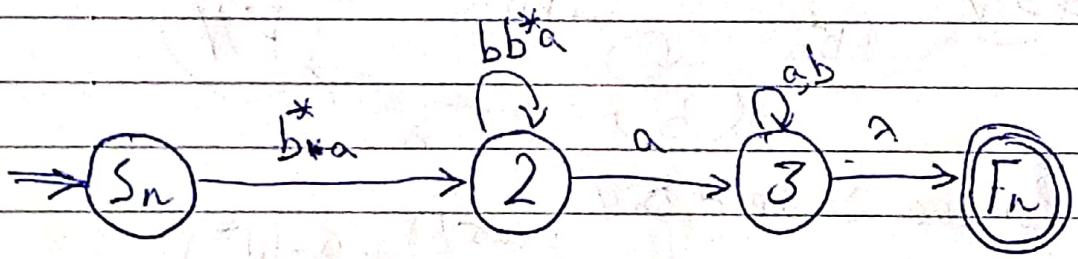
loop:

1 Remove state #1 in=2, out=1

Paths = 2*1 = 2

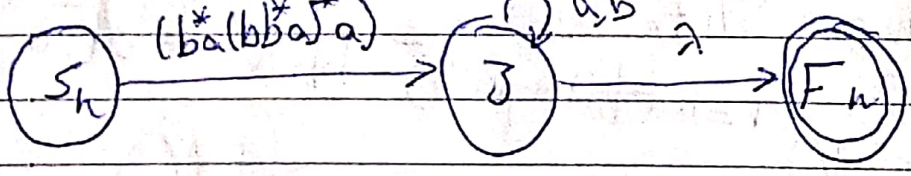
$S_n \rightarrow 1 \rightarrow 2 \; (b^*a)$

$2 \rightarrow 1 \rightarrow 2 \; (bb^*a)$



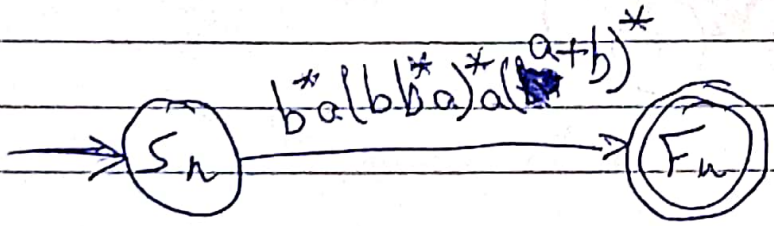
2 Remove state #2 in=1, out=1, Paths=1

$S_n \rightarrow 2 \rightarrow 3 \; (b^*a(bb^*a)^*a)$

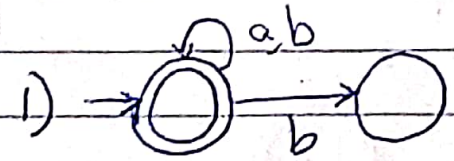


[3] Remove State #3 in=1, out=1, Paths=1

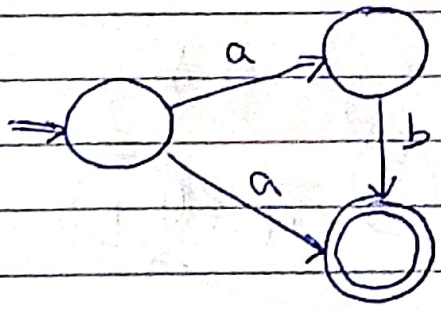
$S_n \longrightarrow 3 \longrightarrow F_n \quad b^*a(bb^*a)^*a(a+b)^*$



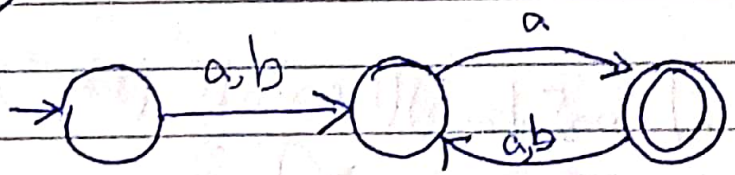
Ex. Convert NFA/DFA to RE

1) 

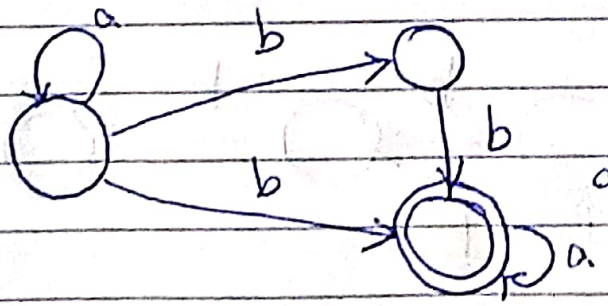
$r = (a+b)^*$

2) 

$r = a + ab$ / OR / $r = a(\lambda + b)$

3) 

$r = ((a+b)a((a+b)a)^*)$

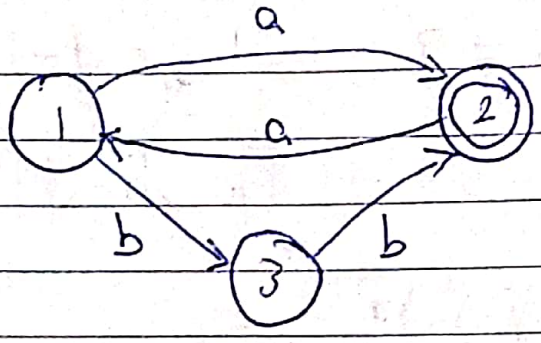
4) 

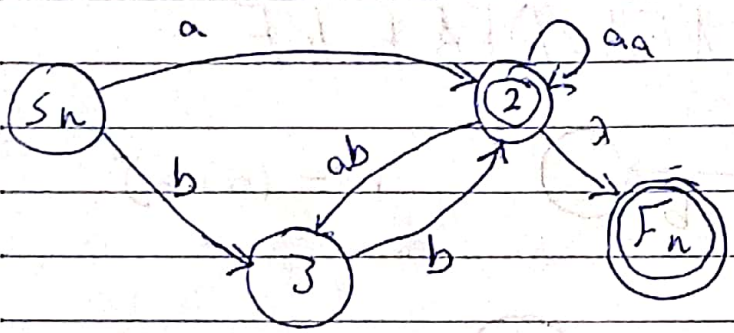
$r = a^*bba^* + a^*ba^*$

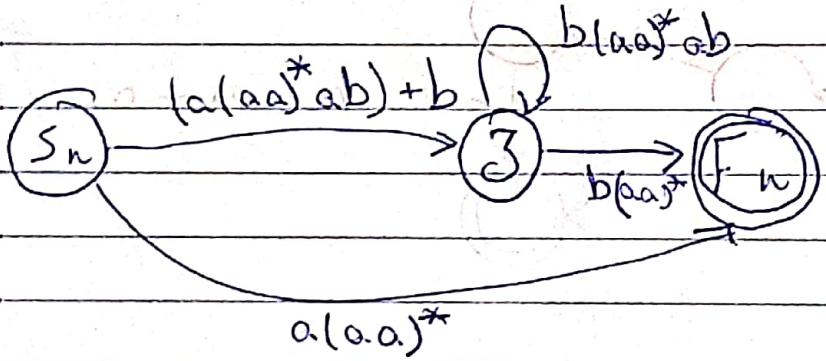
OR

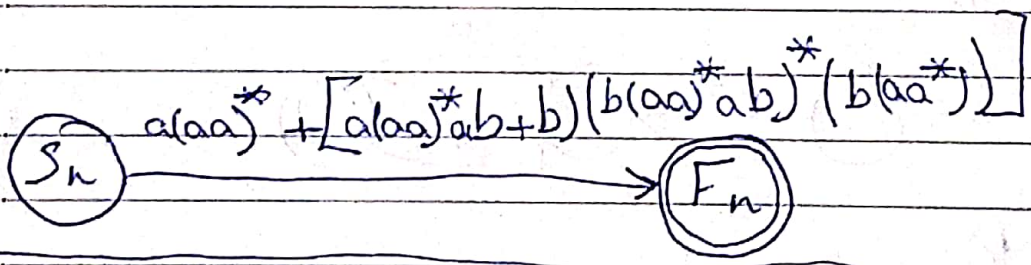
$r = a^*(b+bb)a^*$

5)

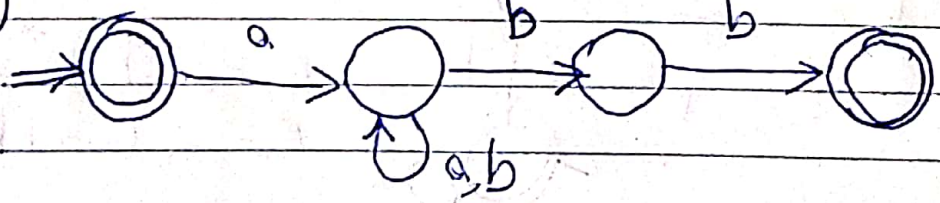






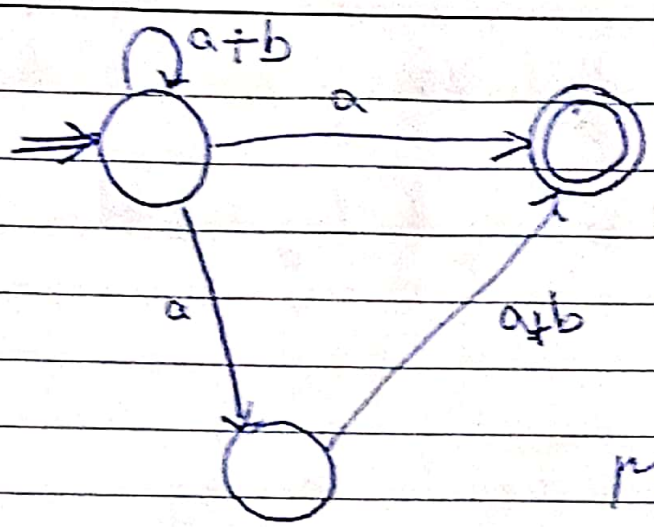


6)



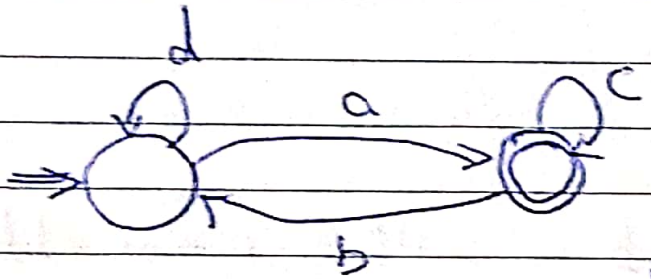
$r = \lambda + (a(a+b)^* bb)$

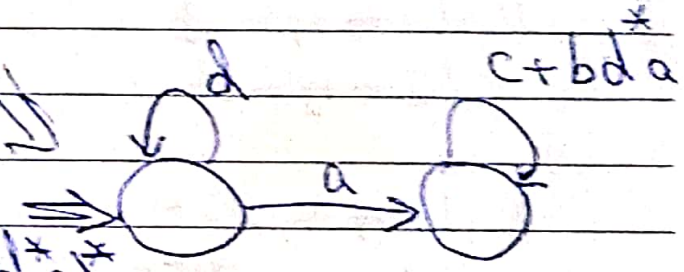
Ex



$r = (a+b)^*(a + a(a+b))$

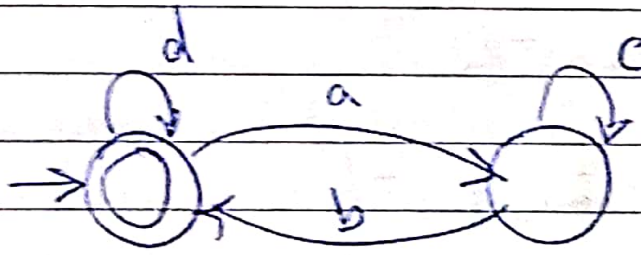
Ex

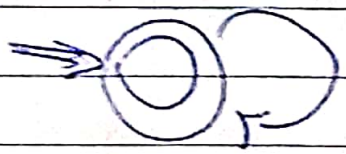




$R = d^* a (c + b d^* a)^*$

Ex.





$R = (d + ac^*b)^*$

* Properties of Regular languages :-

- if L1 and L2 are Regular languages then :-

$L_1 \cup L_2$
$L_1 L_2$
L_1^R
$\overline{L_1}$
$L_1 \cap L_2$

} are Regular Languages

Proof using RE

1) $L_1 \Rightarrow r_1 = ab$ $\quad L_1 \cup L_2 \quad r_3 = ab + bb$
$L_2 \Rightarrow r_2 = bb$

2) $L_1 \Rightarrow r_1 = a^*b$
$L_2 \Rightarrow r_2 = (b+a)^*b$ find $L_1 L_2$

$\Rightarrow r_3 = a^*b(b+a)^*b$

$L_2 L_1 \Rightarrow r_3 = (b+a)^*b\, a^*b$

3) $L_1 \Rightarrow r_1 = ab^*bb\,a$ find r for L_1^R

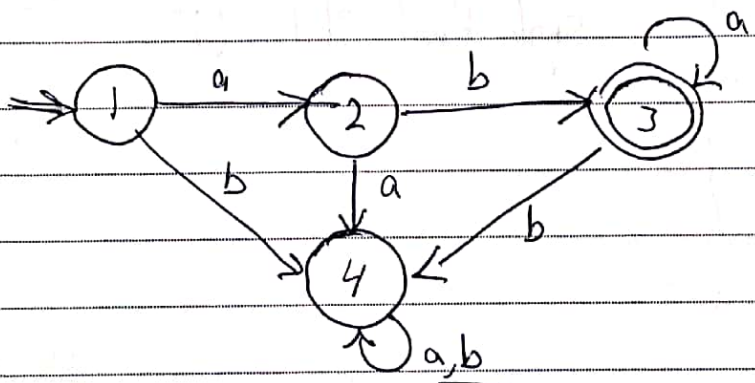
$r = abbb^*a$

Note :- $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$

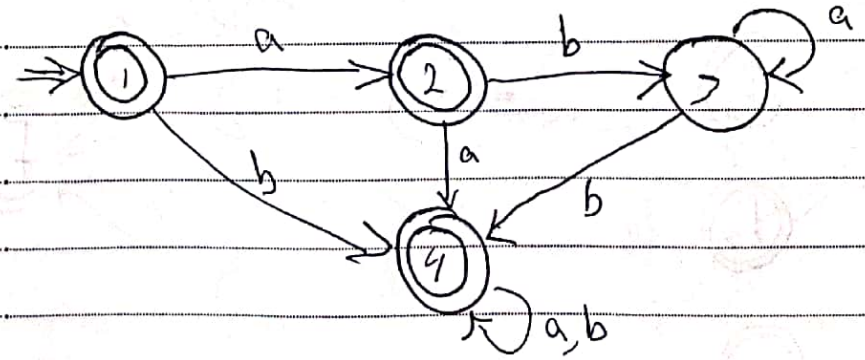
Steps :- DFA For L_1
DFA For L_2
DFA for $\overline{L_1}$
DFA for $\overline{L_2}$
NFA for $\overline{L_1} \cup \overline{L_2}$
DFA for $\overline{L_1} \cup \overline{L_2}$

* Find r_2 for $\overline{L_1}$ $L_1 \Rightarrow r_1 = aba^*$

[1] Construct DFA for r_1



[2] Construct DFA for $\overline{L_1}$



[5] Convert DFA for $\bar{L}$ to Regular expression

$$R_{\text{for } \bar{L}} = \lambda + a + aa(a+b)^* + b(a+b)^*$$

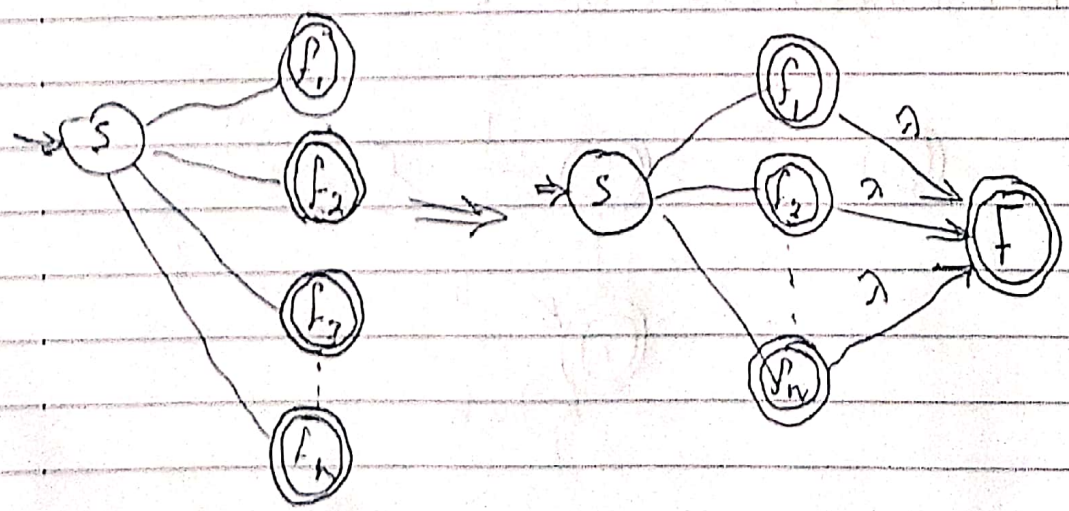
L_1, L_2 are Regular Language

- $L_1 \cap L_2$
- $L_1 \cup L_2$
- L_1^R
- $\bar{L}_1$
- $L_1 L_2$

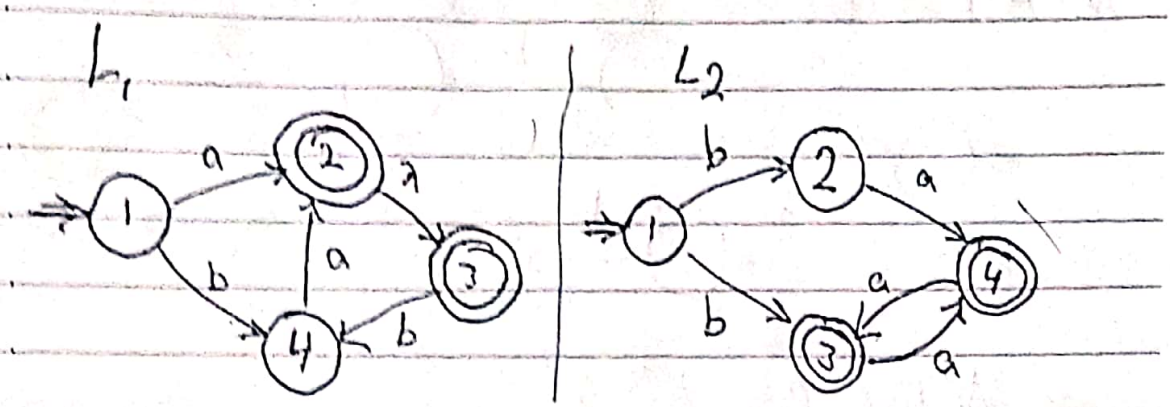
} is Regular

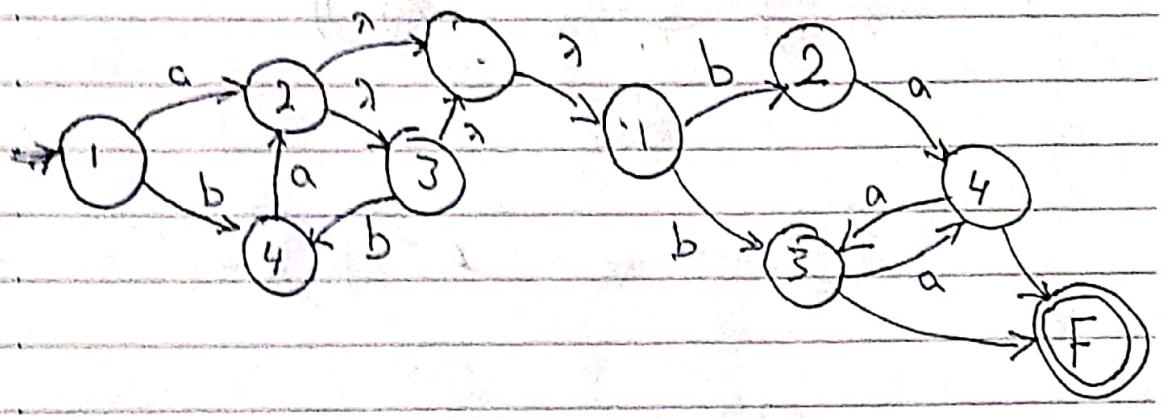
* useful transformation ~~single final~~ single final state (F)

~~Ex~~ Ex.

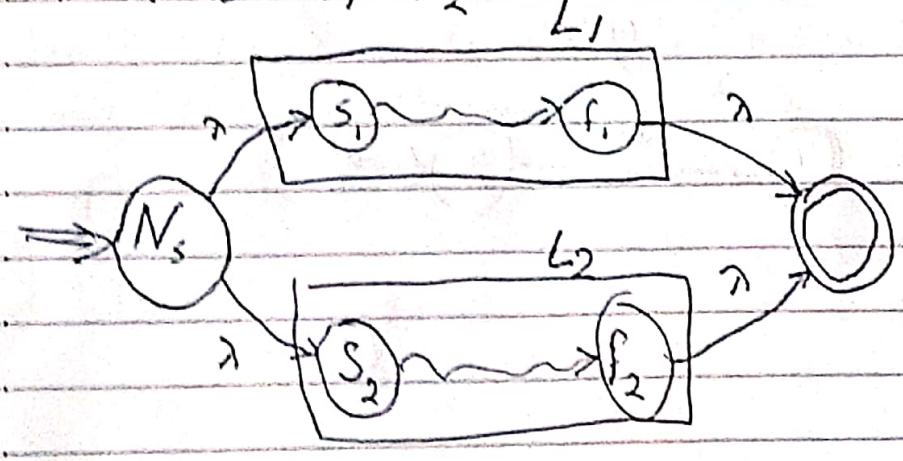


Ex - given L_1 and L_2 find $L_1 L_2$?



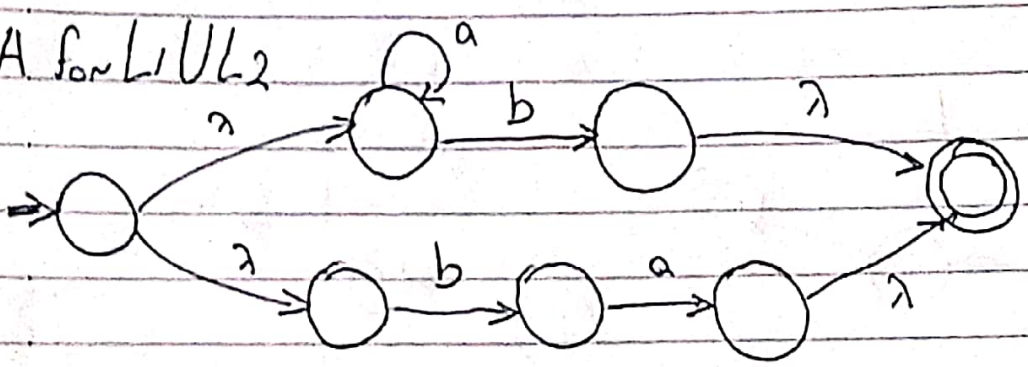


* NFA for $L_1 \cup L_2$



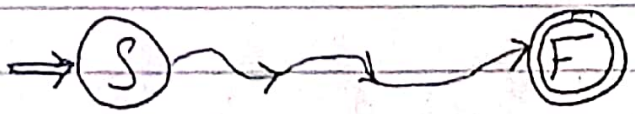
Ex. $L_1 = \{a^*b\}$ $L_2 = \{ba\}$

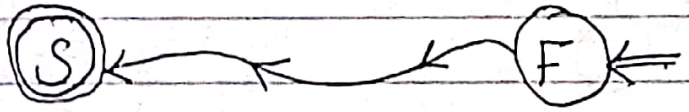
NFA for $L_1 \cup L_2$



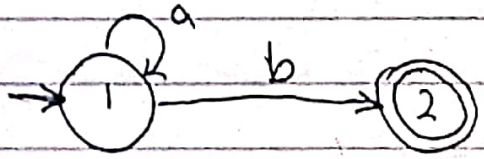
* NFA for L_1^R ?

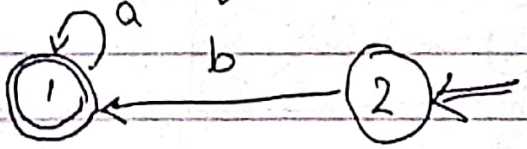
L_1 :





L_1 :





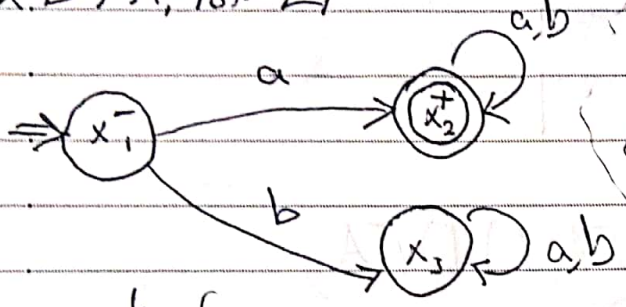
Ex: given

L_1 = all strings that begin with an a.

L_2 = all strings that end with an a.

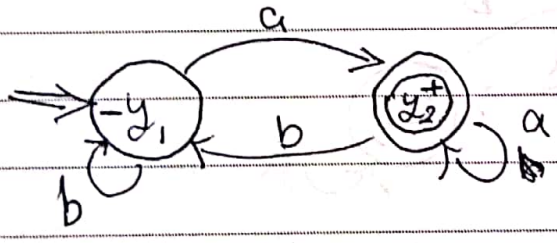
theorem: if L_1 and L_2 are Regular Languages then $L_1 \cap L_2$ is Regular Language

* DFA, for L_1



-: start
+: accept

* DFA_2 for L_2

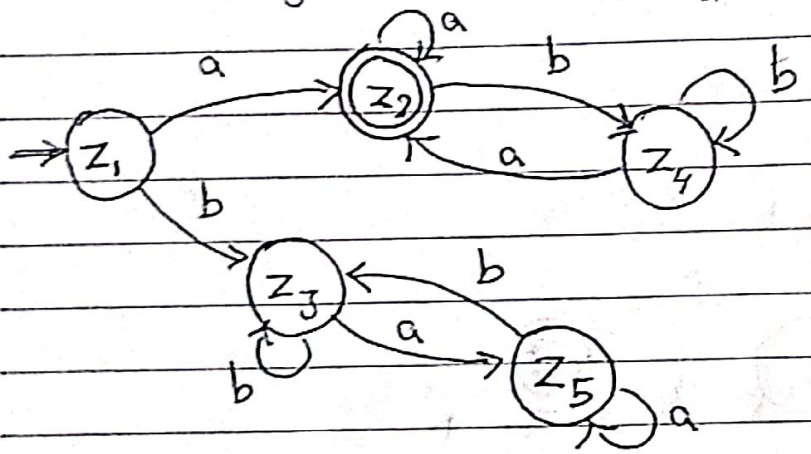


* build transition table of Machine that Runs DFA, and DFA_2 in parallel

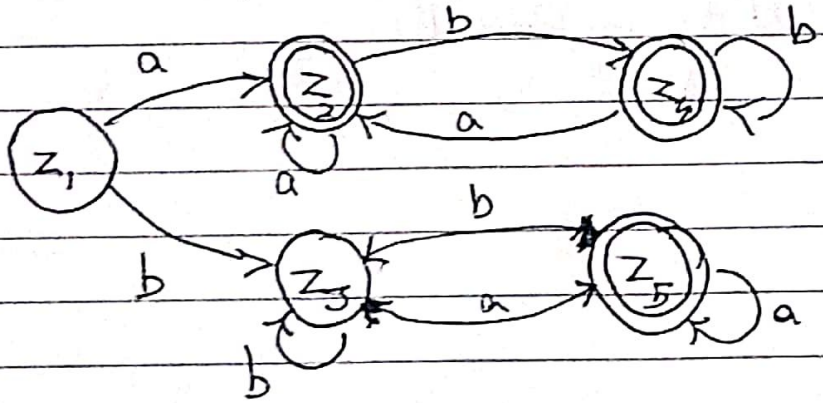
⟹

DFA_3		read a	read b
$-Z_1$	x_1 or y_1	x_2 or y_2	x_3 or y_1
$+Z_2$	x_2 or y_2	Z_2	x_2 or y_1
Z_3	x_3 or y_1	x_3 or y_2	Z_3
Z_4	x_2 or y_1	Z_2	Z_4
Z_5	x_3 or y_2	Z_5	Z_3

* Draw $DFA_3 = DFA_1 \cap DFA_2$

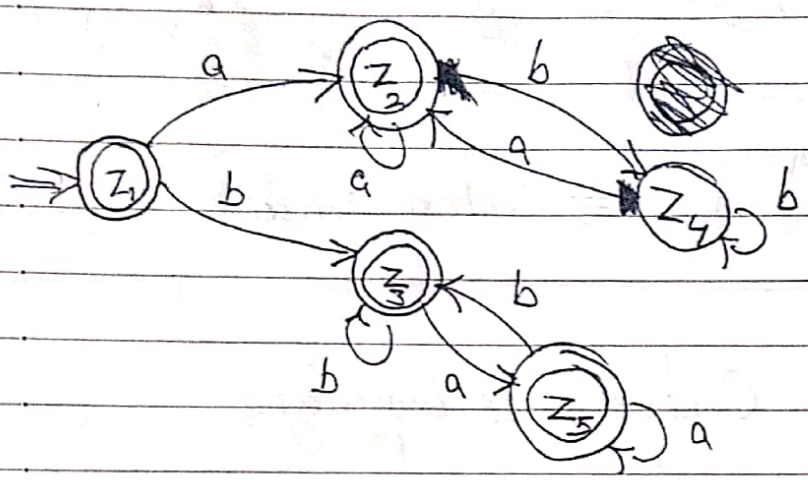


* Find $DFA_4 = DFA_1 \cup DFA_2$



x_1, x_3

* Find DFA = $\overline{DFA_1}$ U DFA_2



* DFA = DFA_1 xor DFA_2

$F = \{Z_4, Z_5\}$

		xor
0	0	0
0	1	1
1	0	1
1	1	0

Context Free Languages

$a = \{\ ,\ b = \}$

$L = \{a^n b^n : n \geq 1\} = \{"\{"^n\ "\}"^n : n \geq 1\}$

$L = \{a^n b^n c^n : n \geq\} \Rightarrow$ Context Sensitive Language

Context Free Language

- Context Free Grammer (C.F.G.)
- Pushdown Automator (PDA)

Expression: $\frac{(1/2)+(9)}{(7)*(\frac{5}{16})}$ (3)

expressions

$\Rightarrow ((1/2)+9)/((7*(5/16))/3)$

* Grammer

$E \rightarrow n$ n = any number ex. $5+5 \Rightarrow n+n \Rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow E*E \Rightarrow 2*3 \Rightarrow n*n \Rightarrow E*E$

$E \rightarrow E/E \Rightarrow$

$E \rightarrow E-E$

$E \rightarrow (E) \Rightarrow (2) \Rightarrow (n) \Rightarrow (E) \Rightarrow E$

def. CFG $G = (T, NT, P, S)$

Terminals (Σ) — Non-Terminals (Variables) — Start NT

$T = \{n, +, *, /, -, (,)\}$

$NT = \{E\}$

P = Set of Production Rules — left hand Side → $P \rightarrow \alpha$ (RHS)

LHS: one NT goes to → sequence of T and NT

$P = \begin{cases} E \to n & (1) \\ E \to E + E & (2) \\ E \to E * E & (3) \\ E \to E - E & (4) \\ E \to E / E & (5) \\ E \to (E) & (6) \end{cases}$

Note * 5-2-1

1. ((5-2)-1) ✓
2. (5-(2-1)) ✗

* Derivation: ($\Rightarrow$)

1. Derive String (expression) $= n*(n+n)/n$

Start NT

$E \overset{*}{\Rightarrow} n*(n+n)/n$

$E \overset{E*E}{\underset{(3)}{\Rightarrow}} \boxed{E} * E$

$\overset{n}{\underset{(1)}{\Rightarrow}} n * \boxed{E}$

$\overset{E/E}{\underset{(4)}{\Rightarrow}} n * \boxed{E} / E$

$\overset{(E)}{\underset{(6)}{\Rightarrow}} n * (\boxed{E}) / E$

$\overset{(2)}{\Rightarrow} n * (\boxed{E} + E) / E$

$\overset{(1)}{\Rightarrow} n * (n + \boxed{E}) / E$

$\overset{(1)}{\Rightarrow} n * (n + n) / E$

$\overset{(1)}{\Rightarrow} n * (n + n) / n$

* derive $E \overset{*}{\Rightarrow} (2*5)-1$

$E \overset{(5)}{\Rightarrow} E-E$
$\overset{(6)}{\Rightarrow} (E)-E$
$\overset{(5)}{\Rightarrow} (E*E)-E$
$\overset{(1)}{\Rightarrow} (n*E)-E$
$\overset{(1)}{\Rightarrow} (n*n)-E$
$\overset{(1)}{\Rightarrow} (n*n)-n$

CFG:-
Ex: $G=(T, NT, P, S)$ $T=\{a\}, NT=\{S\}$

$P=\begin{cases} 1.\ S \rightarrow aS \\ 2.\ S \rightarrow \lambda \end{cases}$ $S=S$

1] derive String "aaa"
$S \overset{*}{\Rightarrow} aaa$
$\overset{(1)}{\Rightarrow} aS$
$\overset{(1)}{\Rightarrow} aaS$
$\overset{(1)}{\Rightarrow} aaaS$
$\overset{(2)}{\Rightarrow} aaa$

2] derive String λ
$S \overset{*}{\Rightarrow} \lambda$
$S \overset{(2)}{\Rightarrow} \lambda$

Ex:2 $G=\{T, NT, P, S\}$

$P=\{$
1. $A \to X$
2. $A \to Y$
3. $X \to aX$
4. $X \to bX$
5. $X \to a$
6. $X \to b$
7. $Y \to \lambda$
$\}$

$T=\{a,b\}$

$NT=\{A, X, Y\}$

$S=A$

String	derivation
λ	$A \overset{*}{\Rightarrow} \lambda$ / $A \overset{(2)}{\Rightarrow} Y \overset{(7)}{\Rightarrow} \lambda$
b	$A \overset{*}{\Rightarrow} b$ / $A \overset{(1)}{\Rightarrow} X \overset{(6)}{\Rightarrow} b$
$ababb$	$A \overset{*}{\Rightarrow} ababa$ / $A \overset{(1)}{\Rightarrow} X \overset{(3)}{\Rightarrow} aX \overset{(4)}{\Rightarrow} abX \overset{(3)}{\Rightarrow} abaX \overset{(4)}{\Rightarrow} ababX \overset{(6)}{\Rightarrow} ababb$

$L(G)=\{w : w \in \{a,b\}^*\}$

* CFG:

$$S \to \alpha_1$$
$$S \to \alpha_2 \Rightarrow S \to \alpha_1 | \alpha_2 | \alpha_3$$
$$S \to \alpha_3$$

3 Production Rules

Example:-

1) $G: S \to aSa | bSb | \lambda$

Find $L(G) = \{\lambda, aa, bb, abba, baab, aaaaaa, abbbba, aabbaa, baaaab, \ldots\}$

$L(G) = \{ww^R : w \in \{a,b\}^*\}$

$L(G) = \{\text{even Palindrome}\}$

2) $G_2: S \xrightarrow{(1)} aSa | bSb | a | b | \lambda$ (1) (2) (3) (4) (5)

$L(G_2) = \{\text{Palindrome}\}$

* derive $S \Rightarrow babab \overset{(2)}{\Rightarrow} bSb \overset{(1)}{\Rightarrow} baSab \overset{(4)}{\Rightarrow} babab$

G_3: $S \to aSa \mid b$

$L(G_3) = \{a^n b a^n : n \geq 0\}$

G_4: $S \to aSb \mid \lambda$

$L(G_4) = \{a^n b^n : n \geq 0\}$

G_5: $S \to aS \mid b$

$L(G_5) = \{a^n b : n \geq 0\}$

G_6: $S \to aaSb \mid c$

$L(G_6) = \{a^{2n} c b^n : n \geq 0\}$

G_7: $S \to XbaY$

$X \to aX \mid \lambda$

$Y \to bbY \mid \lambda$

$L(G_7) = \{a^n b a b^{2m} : n, m \geq 0\}$

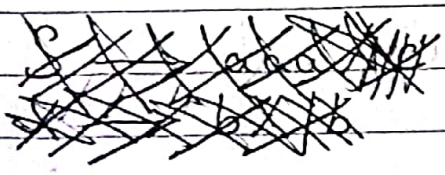
* Write CFG for $L = \{a^n b a b^{2m} : n, m \geq 1\}$

1) G: $S \to XbaY$

$X \to aX \mid a$

$Y \to bbY \mid bb$

2) $L=\{a^{3n}b^{m}c^{n} : n\geq 0, m\geq 1\}$



$S \xrightarrow{(1)} aaaSc \mid X \xrightarrow{(2)}$

$X \xrightarrow[(3)]{} bX \mid \underset{(4)}{b}$

$S \overset{*}{\Rightarrow} aaaaaabbcc$

$S \overset{1}{\Rightarrow} aaaSc \overset{(1)}{\Rightarrow} aaaaaaaScc \overset{2}{\Rightarrow} aaaaaaXcc$

$\overset{(3)}{\Rightarrow} aaaaaaabXcc \overset{(4)}{\Rightarrow} aaaaaaabbcc$

Ex. $L(G)=\{w : S \overset{*}{\Rightarrow} w, w\in T^{*}\}$

Language generated by Grammar G.

start Variable

Terminals (Σ)

Ex. write a grammar for the following language

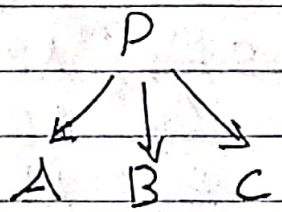
$L=\{a^{n}b^{m} : n>m\}$

$S \Rightarrow aX \mid aS$

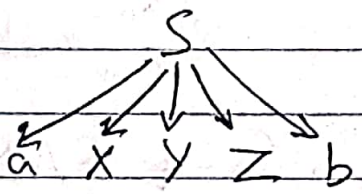
$X \Rightarrow aXb \mid \lambda$

Derivation orders and Derivation Trees

$P \to A\ B\ C$



$S \to a\ X\ Y\ Z\ b$



1. derivation orders
   a) left most derivation ($\Rightarrow_{lm}$, $\Rightarrow$)
   b) Right most derivation ($\Rightarrow_{rm}$)

Ex. (start variable)

G: 1. $S \Rightarrow AB$
2. $A \Rightarrow aaA$
3. $A \Rightarrow \lambda$
4. $B \Rightarrow Bb$
5. $B \Rightarrow \lambda$

use left most derivation to derive String "aab"

$S \overset{*}{\Rightarrow}_{lm} aab$

$S \overset{(1)}{\Rightarrow}_{lm} AB$

$\overset{(2)}{\Rightarrow}_{lm} aaAB$

$\overset{(3)}{\Rightarrow}_{lm} aaB$

$\overset{(4)}{\Rightarrow}_{lm} aaBb$

$\overset{(5)}{\Rightarrow}_{lm} aab$

* Right most

$$S \overset{*}{\Rightarrow} aaab$$

$$S \overset{(1)}{\underset{rm}{\Rightarrow}} A\underline{B}$$

$$\overset{(4)}{\underset{rm}{\Rightarrow}} ABb$$

$$\overset{(5)}{\underset{rm}{\Rightarrow}} Ab$$

$$\overset{(2)}{\underset{rm}{\Rightarrow}} aaAb$$

$$\overset{(3)}{\underset{rm}{\Rightarrow}} aaab$$

Ex. 1. $S \rightarrow SAS$
2. $S \rightarrow b$
3. $A \rightarrow ba$
4. $A \rightarrow \#\#bA$

use right most derivation to derive string "bbbaab"

$$S \overset{*}{\underset{rm}{\Rightarrow}} bbbaab$$

$$S \overset{(1)}{\underset{rm}{\Rightarrow}} SAS$$

$$\overset{2}{\underset{rm}{\Rightarrow}} SAb$$

$$\overset{4}{\underset{rm}{\Rightarrow}} SbAb$$

$$\overset{3}{\underset{rm}{\Rightarrow}} Sbbab$$

$$\overset{2}{\underset{rm}{\Rightarrow}} bbbaab$$

* Derivation orders (leftmost, Rightmost)

- Derivation tree

1. $S \to AB$
2. $A \to aaA \mid \lambda$
3. $B \to Bb \mid \lambda$

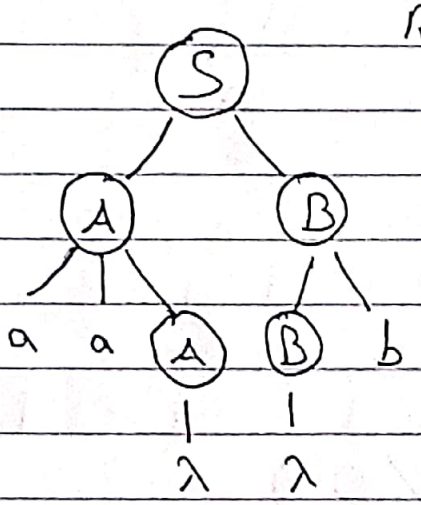
derivation tree of string "aab"

leftmost $S \overset{*}{\Rightarrow} aab$

$S \overset{1}{\Rightarrow} AB \overset{2}{\Rightarrow} aaAB \overset{3}{\Rightarrow} aaB \overset{4}{\Rightarrow} aaBb \overset{5}{\Rightarrow} aab$

Rightmost $S \overset{*}{\Rightarrow} aab$

$S \underset{rm}{\Rightarrow} AB \underset{rm}{\overset{4}{\Rightarrow}} ABb \underset{rm}{\overset{5}{\Rightarrow}} Ab \underset{rm}{\overset{2}{\Rightarrow}} aaAb \underset{rm}{\overset{3}{\Rightarrow}} aab$



Ex. $E \to E+E \mid E*E \mid E/E \mid E-E \mid (E) \mid a$ (integers)

derivation tree of "3*((5+7)/2)"

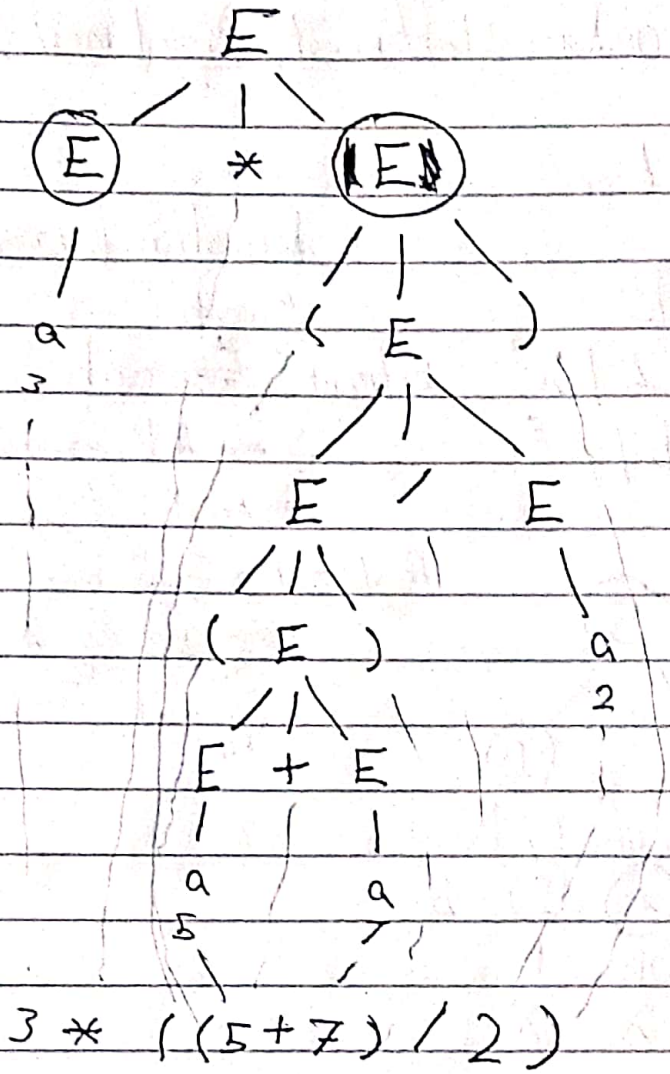
$E \overset{*}{\Rightarrow} 3*((5+7)/2)$

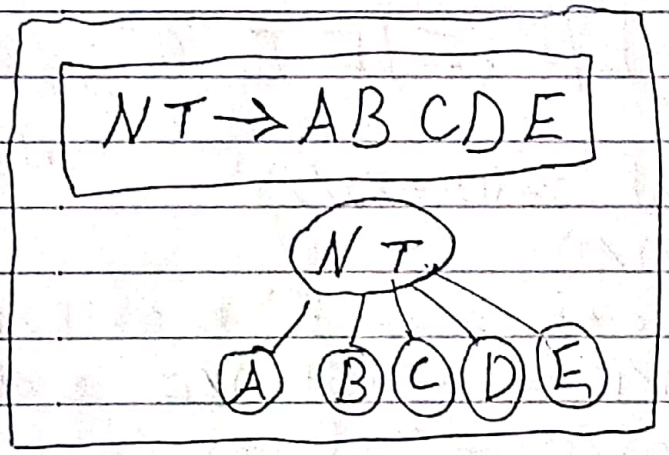
$\Rightarrow E*E \Rightarrow a*E \Rightarrow a*(E) \Rightarrow a*(E/E)$

$\Rightarrow a*((E)/E) \Rightarrow a*((E+E)/E) \Rightarrow a*((a+E)/E)$

$\Rightarrow a*((a+a)/E) \Rightarrow 3*((a+a)/a)$

$\Longrightarrow$

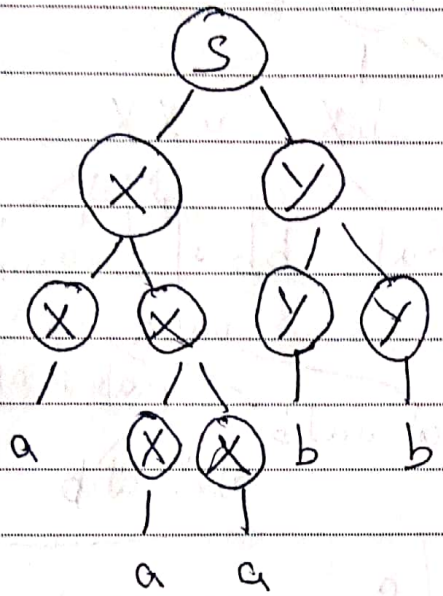

E
E
*
E
a
3
(
E
)
E
E
(
E
)
a
2
E
+
E
a
5
a
3 * ((5+7) / 2)


NT → A B C D E
NT
A
B
C
D
E

* G: $S \to XY$
$X \to XX \mid a$
$Y \to YY \mid b$

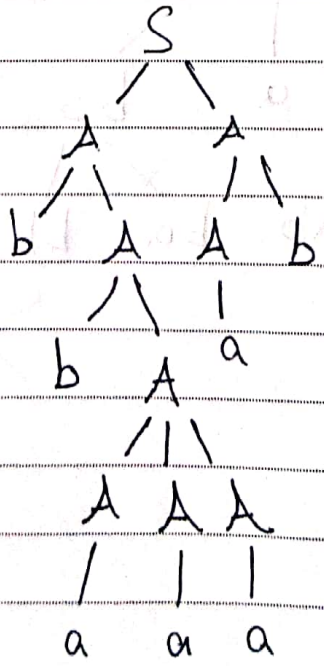
draw Parsetree of word "aaabb"

$S \overset{*}{\Rightarrow} aaabb$
$\Rightarrow XY \Rightarrow XXY \Rightarrow aXY$
$\Rightarrow aXXY \Rightarrow aaXY \Rightarrow aaaY$
$\Rightarrow aaaYY \Rightarrow aaabY$
$\Rightarrow aaabb$



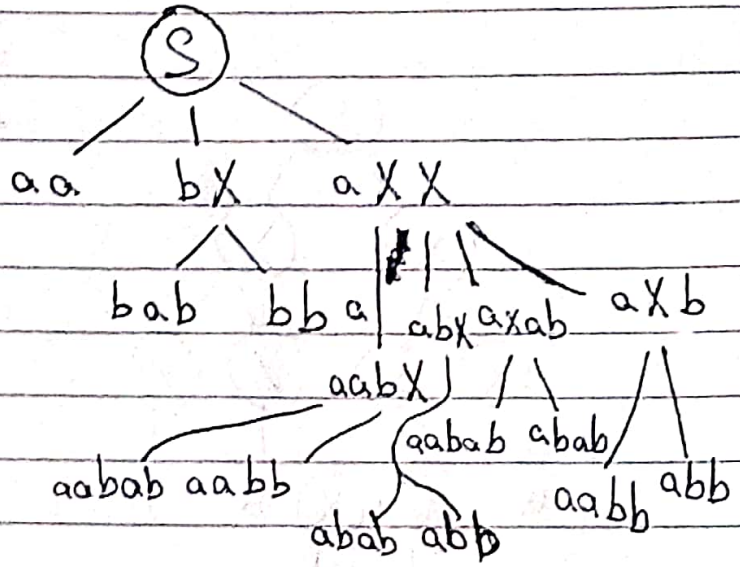
* G: $S \to AA$
$A \to AAA \mid bA \mid Ab \mid a$

draw Parsetree of word "bbaaaab

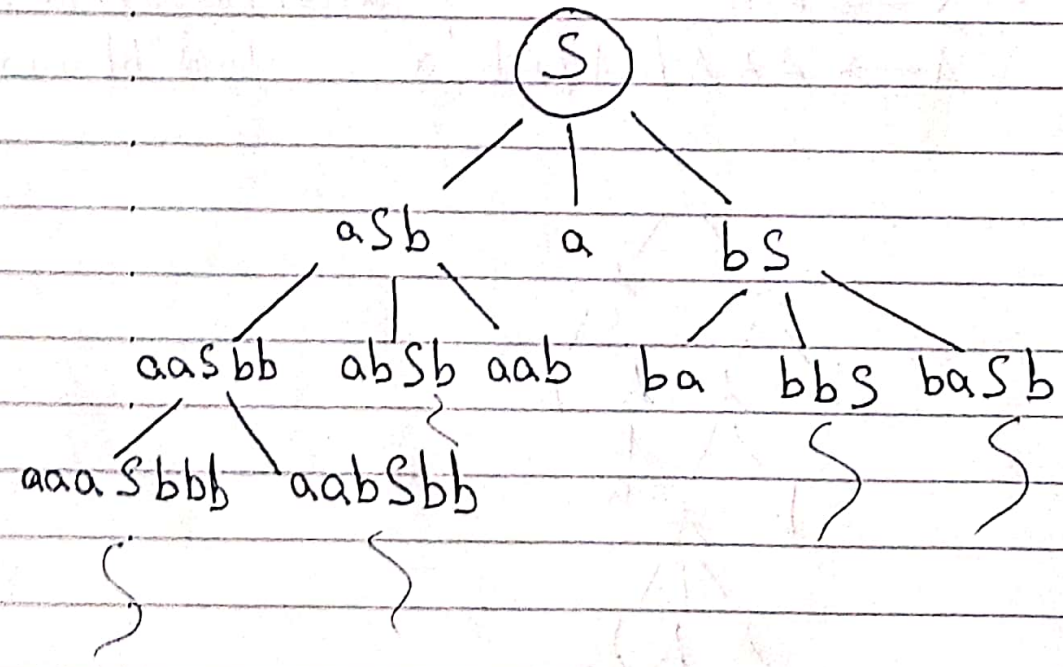


* the total language Tree :-

G : S → aa | bX | aXX
X → ab | b

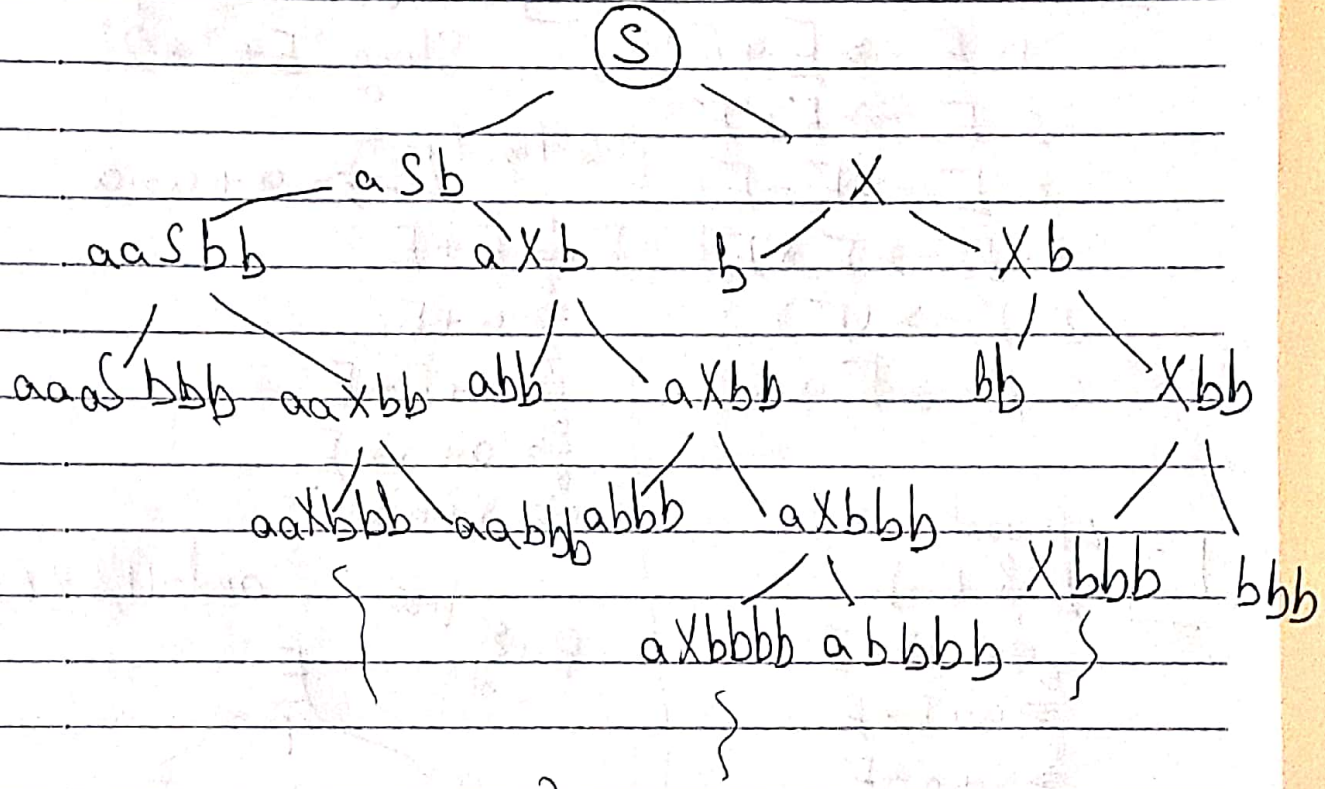


G : S → aSb | bS | a



$G: S \rightarrow aSb \mid X$

$X \rightarrow Xb \mid b$



$S \rightarrow aSb \mid X$

$X \rightarrow Xb \mid b$

$\Rightarrow L(G) = \{ a^n b^m : m > n \}$

* Ambiguity :-

Ex G:

1. $E \to E + E$
2. $E \to E / E$
3. $E \to E - E$
4. $E \to E * E$
5. $E \to (E)$
6. $E \to \$a$

[1] derivation tree of String "5+3*2"

leftmost $E \overset{*}{\Rightarrow} a+a*a$

$E \overset{1}{\Rightarrow} E+E$

$\overset{6}{\Rightarrow} a+E$

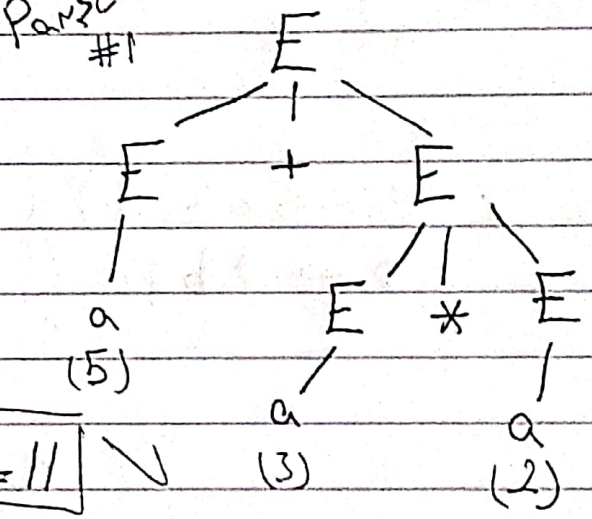
$\overset{4}{\Rightarrow} a+E*E$

$\overset{6}{\Rightarrow} a+a*E$

$\overset{6}{\Rightarrow} a+a*a$

order(1,6,4,6,6)

Parse tree #1



V = 11 ✓

[2] leftmost

$E \overset{4}{\Rightarrow} E*E$

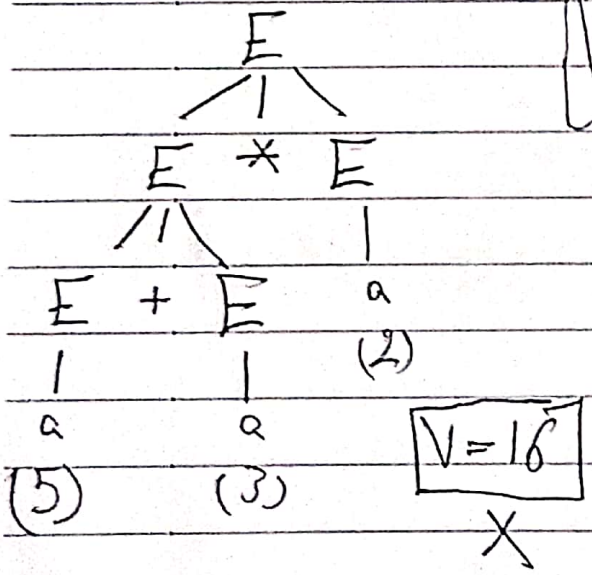
$\overset{1}{\Rightarrow} E+E*E$

$\overset{6}{\Rightarrow} a+E*E$

$\overset{6}{\Rightarrow} a+a*E$

$\overset{6}{\Rightarrow} a+a*a$

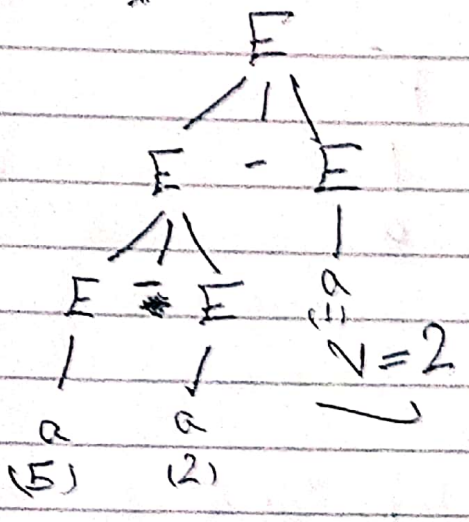
Parse tree #2



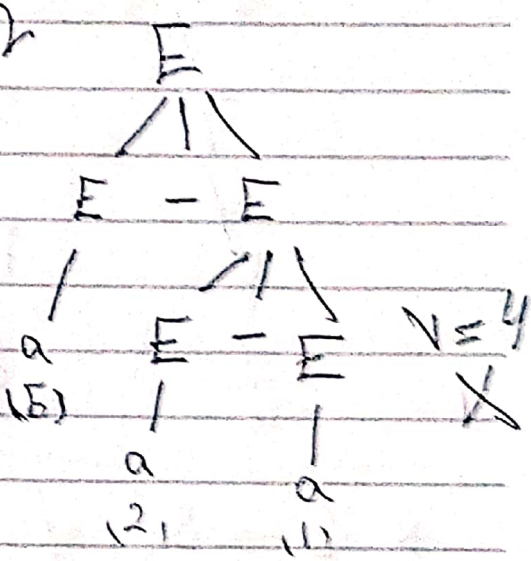
V = 16 ✗

* "5-2-1"

Parse tree #1



Parse tree #2



* ambiguous CFG:

$$E \to E+E \mid E*E \mid (E) \mid a$$

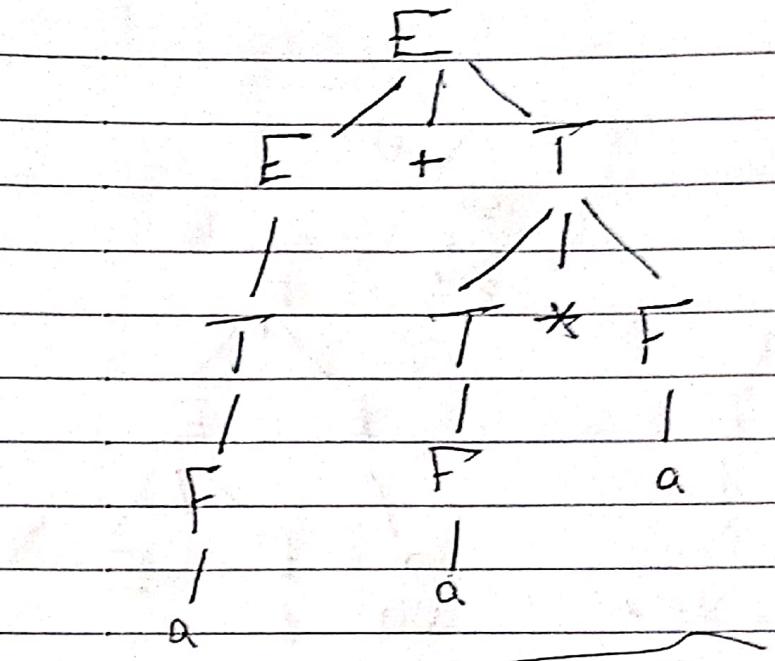
string a+a*a — Parse tree #1
Parse tree #2

* Unambiguous Grammar

$$E \to E+T^{1} \mid T^{2}$$
$$T \to T*F^{3} \mid F^{4}$$
$$F \to (E)^{5} \mid a^{6}$$

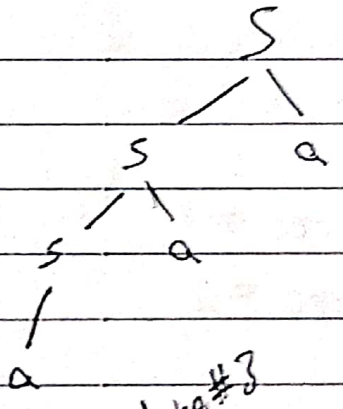
One derivation tree only for "a+a*a"

$$E \overset{1}{\Rightarrow} E+T \overset{2}{\Rightarrow} T+T \overset{4}{\Rightarrow} F+T \overset{6}{\Rightarrow} a+T \overset{3}{\Rightarrow} a+T*F \overset{4}{\Rightarrow} a+F*F \overset{6}{\Rightarrow} a+a*F \overset{6}{\Rightarrow} a+a*a$$

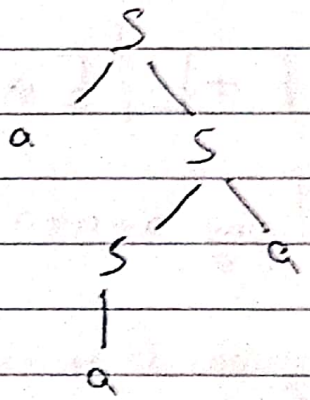


* $S \rightarrow aS \mid Sa \mid a$ String "aaa"

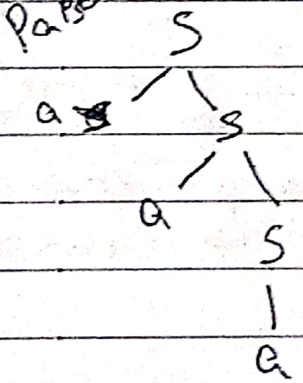
Parse tree #1



Parse tree #2



Parse tree #3



ambiguous

Unambiguous Grammar :- $S \rightarrow aS \mid a$

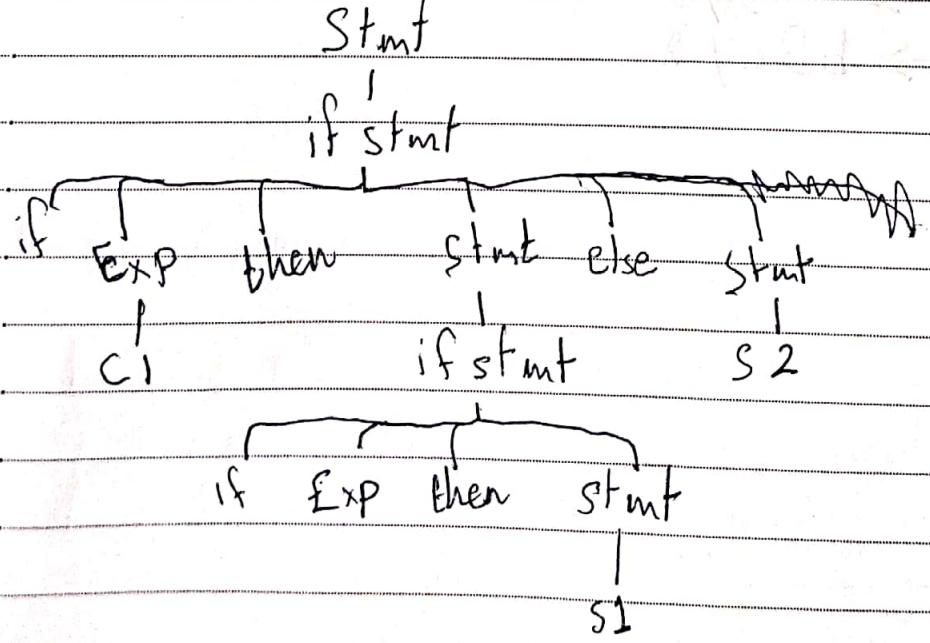
Ex $S \rightarrow SaSaS \mid b$

h.m

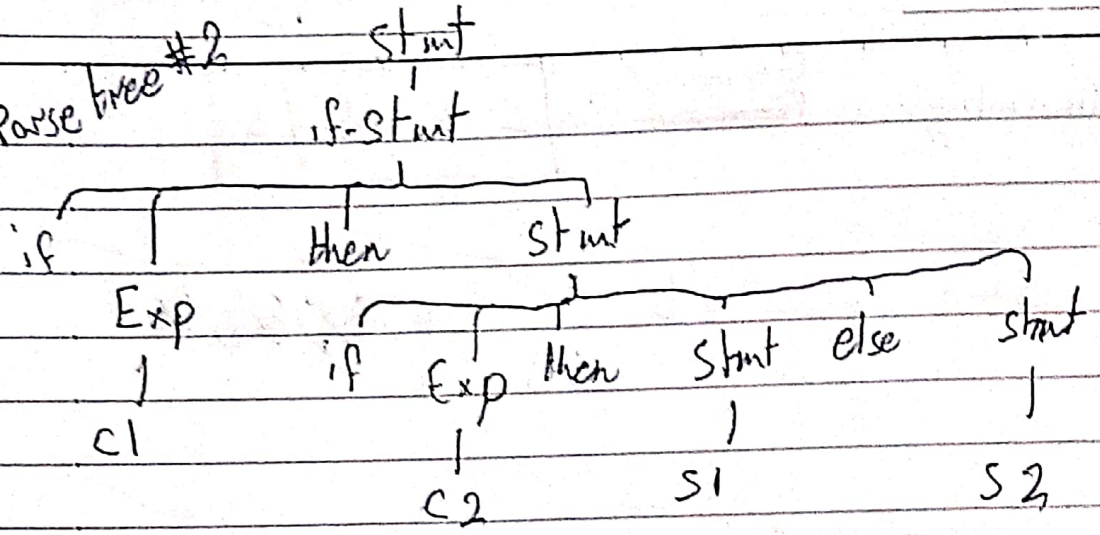
* if-stmt → if Exp then stmt
  if-stmt → if Exp then stmt else stmt
  stmt → if-stmt / s1 / s2
  Exp → c1 / c2

b) if c1 then if c2 then s1 else s2

Parse tree #1



Parse tree #2



$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$ $n, m \geq \emptyset$

* من الامثلة اللي دائماً ambiguous $S_1 \rightarrow S_1 \mid S_2$

$S_1 \rightarrow S_1 c \mid A$
$A \rightarrow a A b \mid \lambda$
$S_2 \rightarrow a S_2 \mid B$
$B \rightarrow b B c \mid \lambda$

Simplifications of CFGs

I. substitution Rule

$S \to aB$

$A \to aaA$

$A \to abBc$

$B \to aA$

$B \to b$

Substitute $B \to b$

$S \to ab \mid aB$

$A \to abbc \mid aaA \mid abBc$

$B \to aA$

Substitute $B \to aA$

$S \to ab \mid aaA$

$A \to abbc \mid aaA \mid abaAc$

* In general $A \to xBz$

$B \to y$

substitute $B \to y$

$A \to xBz$

$A \to xyz$

Nullable variables

~~Nullable~~

$X \to \lambda$ ← Nullable Production

Ex.

$S \to aMb$

$M \to bMa$

$M \to \lambda$ ⟹ Substitute $M \to \lambda$

$S \to aMb$

$S \to ab$

$M \to bMa$

$M \to ba$

Unit Productions

$X \to Y$

1 Variable

Ex $S \to aA$

$A \to a$

$A \to B$ ← unit production

$B \to A$

$B \to bb$

⟹ remove $A \to B$ by substitution

$S \to aA \mid aB$

$A \to a$

$B \to A$

$B \to B$ remove $B \to B$

$B \to bb$

remove $B \to A$ by Substitution ⟹

$S \to aA \mid$ ~~aA~~ ~~aB~~ aB

$A \to a$

$B \to bb$

* Useless Productions

[1] $S \to aSb$

$S \to \lambda$

$S \to A$

$A \to aA$ ← useless Production

$S \to aSb \Rightarrow aAb \Rightarrow aaAb \ldots \infty$

[2] $S \to A$ — useful Variable

$A \to aA$

$A \to \lambda$

$B \to bA$ — useless Production

useless Variable

$S \to A \to aA \to aaA \ldots$ ✗

Ex: $S \to aSb$

useful Variable $S \to \lambda$

$S \to A$

useless Variables $A \to aA$

$B \to C$

$C \to D$

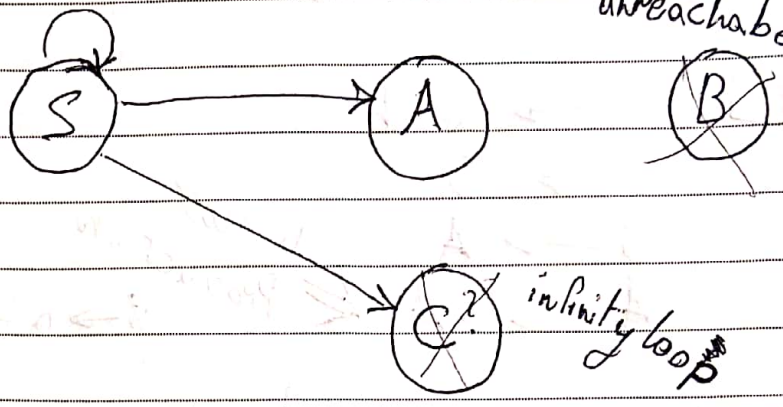
useless Productions

Ex. $S \to aS \mid A \mid C$

$A \to a$

$B \to aa$ Remove

$C \to aCb$ Remove



$S \to aS \mid A$

$A \to a$

Chomsky Normal Form

$A \to BC$ or $A \to a$

① ② ① Terminal (1)

$S \to AS$
$S \to a$
$A \to Sa$
$A \to b$

$S \to AS$
$S \to AAS$
$A \to SA$
$A \to aa$

Ex. $S \to ABa$
$A \to aab$
$B \to Ac$

// → new Variables T_a, T_b, T_c

① $S \to A[BT_a]$
$A \to T_a[T_aT_b]$
$B \to AT_c$
$T_a \to a$
$T_b \to b$
$T_c \to c$

$\Rightarrow$

$S \to AV_1$
$V_1 \to BT_a$
$A \to T_aV_2$
$V_2 \to T_aT_b$
$B \to AT_c$
$T_a \to a$
$T_b \to b$
$T_c \to c$

Ex $S \to aSbSb$

$S \to abb$

① T_a, T_b

$S \to T_a S T_b S T_b$

$S \to T_a T_b T_b$

$T_a \to a$

$T_b \to b$

$\Rightarrow$

$S \to T_a V_1$

$V_1 \to S V_2$

$V_2 \to T_b V_3$

$V_3 \to S T_b$

$S \to T_a V_4$

$V_4 \to T_b T_b$

$T_a \to a$

$T_b \to b$

$A \to a V_1 V_2 V_3 \cdots V_K \quad K \geq 0$

terminal — Variables